```
// Cloud scanner for the US stock signal board. Runs on GitHub Actions (free), so
// It reuses the exact rules inside index.html, writes data/signals.json and an a
// Read only: it holds no broker login and cannot trade.
import fs from 'node:fs';
import os from 'node:os';
import path from 'node:path';
import crypto from 'node:crypto';
import zlib from 'node:zlib';
import { createRequire } from 'node:module';

const env = process.env, MODE = env.MODE === 'intraday' ? 'intraday' : env.MODE =
const local = (u) => u && /^http:\/\/(127\.0\.0\.1|localhost)(:\d+)?$/.test(u) ?
const FH = local(env.FH_BASE) || 'https://finnhub.io/api/v1', FMPB = local(env.FM
const MV_HOSTS = local(env.MASSIVE_BASE) ? [local(env.MASSIVE_BASE)] : ['https://
const TD = local(env.TD_BASE) || 'https://api.twelvedata.com', TG = local(env.TG_
if (!env.TWELVE_KEY) { console.error('Missing secret TWELVE_KEY'); process.exit(1
// SEC EDGAR (official, free; the SEC asks every program to name itself) and Clin
const SEC_DATA = local(env.SEC_BASE) || 'https://data.sec.gov', SEC_WWW = local(e
let SEC_LAST = null;
async function secJson(url) { try { await new Promise(r => setTimeout(r, FAST ? 0
// one SEC filing -> its event families (same table as events-study.mjs, read fro
function evFams(form, items, sic, ES) { const map = (ES && ES.map && ES.map.item)
  if (f === '8-K') for (const it of String(items || '').split(',').map(x => x.tri
  if (f === 'SC 13D' || f === 'SCHEDULE 13D') o.add('ACTIVIST'); if (f === '424B4
  if (o.has('EARN')) { o.delete('NEWS'); o.delete('BIO_NEWS'); } return [...o]; }
// the session a filing reacts in (New York time): before the close -> that day,
function evReact(p) { const d = p.slice(0, 10), wd = new Date(d + 'T12:00:00Z').g
  if (wd === 0 || wd === 6) return nx(d); return p.length >= 16 && p.slice(11, 16
// recent events on a stock that the Sunday study proved to be followed by losses
function eventWarn(sym) { const ES = readJson0('data/events-study.json'), ER = re
  const hits = (ER.events || []).filter(e => e.sym === sym).flatMap(e => e.f.spli
  if (!hits.length) return ''; const h = hits[0], F = ES.fam[h.k];
  return ' | NEWS WARNING ' + h.d + ': ' + F.name + '; after such filings stocks
function newsText() { let L = []; try { L = fs.readFileSync('data/news-log.jsonl'
  if (!L.length) return 'NEWS WATCH: no threat headline about your stocks or toda
  return ['NEWS WATCH: the last threat headlines (heads-up only, words matched, n
function fastText() { const F = readJson0('data/fastmover-lab.json'), V = readJso
  if (!F || !F.rules) return 'FAST MOVERS: the five-year fast-mover lab runs on S
  const ln = (x) => !x || !x.n ? 'no trades' : x.n + ' trades, ' + Math.round(x.w
  const D = F.decode || {}, rec = (Pp && Pp.record) || {};
  return ['FAST MOVERS AND DAY TRADES (five years, whole market incl. 1-3 USD sto
    'RULES (newer half; "best moment" = the average high of the day, what a chart
    '', 'WHAT CAME BEFORE THE JUMPS (found on old data, checked on unseen data):'
    ...(D.chosen ? ['Best pattern by profit: ' + D.chosen.what + ' -> ' + D.chose
```

```
    '', V && V.list && V.list.length ? 'TODAY'S DAY-TRADE LIST (' + V.day + '): '
    ...(() => { const IL = readJson0('data/ignition-lab.json'), IR = readJson0('d
      const rr = (IR && IR.record) || {};
      return ['', 'IGNITION (buying WHILE a stock starts to run; minute-by-minute
        ...Object.entries(rr).filter(([k]) => /^IG8\//.test(k)).map(([k, r]) => '
        ...(IL.variants ? [IL.variants + ' versions tested (versions with no trad
        ...(LV ? ['Last watcher day ' + LV.day + ': ' + (LV.triggers || []).lengt
          ...(LV.coverage ? ['COVERAGE (measured): ' + LV.coverage.minutesWatched
            'Runners (15%+): ' + LV.coverage.moversLiveBeforePlus5 + ' visible li
          ...(LV.missed && LV.missed.length ? ['MISSED-MOVER AUDIT (' + LV.missed
    'Only rules that PASSED send a morning list or a live alert. Use 1,000 USD at
// ECONOMIC CALENDAR (calendar.mjs): CPI, PPI, retail sales, jobs, the Fed... thi
// holidays; ex-dividend days of your stocks. Read from the ENCRYPTED data/calend
function calRead0() { const k = privKey(); if (!k || !fs.existsSync('data/calenda
const CAL_NAME = { CPI: 'Inflation (CPI)', CORE_CPI: 'Core inflation (CPI)', PPI:
function calendarText(short) {
  const C = calRead0(); if (!C) return short ? '' : 'CALENDAR: not loaded yet. It
  const kw = (t) => { const d = new Date(t + (String(t).length <= 19 ? 'Z' : ''))
  const today = new Date().toISOString().slice(0, 10), wk = new Date(Date.now() +
  const INDX = (C.study && C.study.industry) || {}, hasFind = (e) => { const k =
  const evAll = (C.econ || []).filter(e => e.t.slice(0, 10) >= new Date(Date.now(
  const ev = evAll.filter(e => short ? (e.kind || hasFind(e)) : true);
  const v = (x, u) => x === null || x === undefined ? 'n/a' : x + (u && u.length
  const tested = (e, s) => { const r = St && St[e.kind + (s > 0 ? '_HOT' : '_COOL
  const line = (e) => { const done = e.act !== null && e.act !== undefined, bs =
    return (done ? '' : kw(e.t) + ': ') + nm + (e.name && e.name.toLowerCase() !=
  const hol = (C.holidays || []).filter(h => h.d >= today && h.d <= new Date(Date
  const div = Object.entries(C.div || {}).flatMap(([s, a]) => a.filter(x => x.ex
  const IND = C.study && C.study.industry, GRP = { CPI: 'INFLATION', CORE_CPI: 'I
  const prepNames = [...new Set(evAll.filter(e => (e.act === null || e.act === un
    return 'PREPARED for ' + LB.name + ': if above ' + LB.basis + ' -> ' + f(h) +
  const prep = [...new Set(ev.filter(e => e.act === null || e.act === undefined).
    return 'PREPARED for ' + G.toLowerCase() + ' news: if ' + (G === 'INFLATION'
  const anyPass = St && Object.values(St).some(x => /both halves/.test(x.verdict)
  const indPass = IND && Object.values(IND).some(L => (L.up || []).length || (L.d
  const decision = anyPass || indPass ? 'DECISION: only tested reactions change A
  if (short) { if (!ev.length && !hol.length && !div.length) return ''; return ['
  return ['ECONOMIC CALENDAR (US, next 7 days, every release big and small; TipRa
    ...(IND ? [...prep, ...prepNames, ''] : ['What goes up or down after each kin
    'Updated ' + String(C.updated || '').slice(0, 16).replace('T', ' ') + ' UTC.
// MACRO REACTION (calendar study): after a release surprise (hot / cool inflatio
// beat or lagged the market over the next 5 sessions in BOTH halves of the past.
const CAL_GROUP = { CPI: 'INFLATION', CORE_CPI: 'INFLATION', PPI: 'INFLATION', PC
const CAL_LABEL = { INFLATION_HOT: 'HOT INFLATION', INFLATION_COOL: 'COOL INFLATI
let MACRO_C;
const calKey = (name) => String(name || '').toLowerCase().replace(/\(.*?\)/g, ' '
function macroNow() {
```

```
  if (MACRO_C !== undefined) return MACRO_C; MACRO_C = []; const C = calRead0();
  const since = new Date(Date.now() - 7 * 864e5).toISOString(), nowIso = new Date
  for (const e of [...(C.econ || []), ...(C.hist || [])]) { if (e.act === null ||
    const s = Math.sign((e.act - base) * (/claims|unemployment|jobless/i.test(e.n
    for (const k of [e.key || calKey(e.name), CAL_GROUP[e.kind]].filter(Boolean))
  const latest = {}; for (const x of Object.values(byK)) if (!latest[x.k] || late
  const CK = readJson0('data/sec-ciks.json') || {};
  for (const x of Object.values(latest)) { const sg = Math.sign(x.core !== null ?
    const lb = LBL[x.k] || { name: x.k, basis: 'forecast' }, fam = CAL_LABEL[key]
    MACRO_C.push({ key, label: fam || (lb.name + ' ' + (sg > 0 ? 'ABOVE' : 'BELOW
      ev: x.ev.slice(0, 3).map(e => e.name + ' ' + e.act + ' vs ' + (e.fc !== nul
      up: L.up || [], down: L.down || [], events: L.events, upSet: new Set((L.up
  MACRO_C.sort((p, q) => (q.up.length + q.down.length) - (p.up.length + p.down.le
function macroLines(pos, cands) {
  const M = macroNow(), out = []; const nm = (r) => r.name + ' (' + pcs(r.newer.e
  for (const m of M.slice(0, 4)) { out.push('AFTER ' + m.label + ' (' + m.d + ':
    out.push(m.up.length ? '  HUNT, industries that beat the market next: ' + m.u
    out.push(m.down.length ? '  AVOID, industries that lagged: ' + m.down.slice(0
    const heldBad = Object.keys(pos || {}).filter(s => m.downSet.has(m.indOf(s)))
    if (heldBad.length) out.push('  YOUR STOCKS in lagging industries: ' + heldBa
    if (heldGood.length) out.push('  YOUR STOCKS in industries that tend to gain
  return out; }
// YOUR SKIPS vs THE SYSTEM: every buy you skipped (Skipped button), why, and wha
// your tips and your judgment beat the system's list, or the other way round.
async function skipsText() {
  let L = []; try { L = fs.readFileSync('data/skipped.jsonl', 'utf8').trim().spli
  const since = Date.now() - 45 * 864e5, why = {}; for (const r of L) if (r.why)
  const sk = L.filter(r => !r.why && r.ts >= since && r.px > 0), seen = new Set()
  for (const r of sk.reverse()) { if (seen.has(r.sym)) continue; seen.add(r.sym);
  if (!rows.length) return 'YOUR SKIPS: none recorded yet with a price. Tap Skipp
  const NAME = { tip: 'tip said wait', halal: 'not halal / unsure', money: 'no mo
  const byWhy = {}; for (const r of done) (byWhy[r.why] = byWhy[r.why] || []).pus
  return ['YOUR SKIPS (last 45 days): what the price did after you skipped', '',
    'Average since skipping: ' + (avg === null ? 'n/a' : pcs(avg)) + (avg === nul
    ...Object.entries(byWhy).map(([w, a]) => '  ' + (NAME[w] || w) + ': ' + a.len
    'Too few skips to judge until about 20. The system's own record: send  record
// Research registry: every idea ever tested, its result and how many versions we
const PENDING = [
  ['Short interest (FINRA, twice a month)', 'free public files with an archive, s
  ['Big-fund holdings (SEC 13F, quarterly, 45 days late)', 'free SEC data sets go
  ['Options activity', 'paid feed; only after short interest or 13F shows somethi
  ['Why-I-skipped log + why-it-fired log', 'cheap; next after Sunday results'],
  ['Rejected as near copies (do not rebuild)', 'quiet heavy volume / narrow range
];
function experimentsText() {
  const rows = [], add = (where, name, verdict, tried, why) => rows.push({ where,
  const J = (f) => readJson0('data/' + f);
```

```
  const HL = J('hunting-lab.json'); for (const r of (HL && HL.rules) || []) add('
  const WS = J('winners-study.json'); if (WS) add('winners study', 'what winners
  const LS = J('leaders-study.json'); if (LS) { add('leaders study', 'LEADERS (st
  const SC = J('score-study.json'); if (SC) add('score study', '0-100 stock score
  const SM = J('smart-money-study.json'); if (SM) { add('smart money', 'insider b
  const ES = J('events-study.json'); if (ES && ES.fam) { for (const x of Object.v
  else add('company news', 'SEC/FDA news families, supply chain, peer lag, Friday
  { const CS = (calRead0() || {}).study; if (CS && CS.industry) { const L = Objec
  const OC = J('octopus.json'); if (OC && OC.status === 'OK') add('octopus (many
  const FL = J('fastmover-lab.json'); if (FL && FL.rules) for (const r of Object.
  const IG = J('ignition-lab.json'); if (IG && IG.rules) { const v = Object.value
  const N = rows.reduce((a, r) => a + r.tried, 0), cnt = (v) => rows.filter(r =>
  const icon = (v) => /PROMISING/.test(v) ? 'PASS' : v === 'WARNING' ? 'WARN' : v
  const by = {}; for (const r of rows) (by[r.where] = by[r.where] || []).push(r);
  return ['RESEARCH REGISTRY: every idea tested, result, versions tried', '',
    rows.length + ' ideas, ' + N + ' versions tried in total. Passed: ' + (cnt('P
    'The more versions tried, the more one passes by luck. With ' + N + ' tries,
    ...Object.entries(by).flatMap(([w, a]) => [w.toUpperCase() + ':', ...a.map(r
    'WAITING / TO COLLECT (not forgotten):', ...PENDING.map(([a, b]) => '- ' + a
    '', 'Research only. Nothing here is a buy until it passes the paper record.']
function eventsText() { const ES = readJson0('data/events-study.json'); if (!ES |
  const ln = (s) => !s || !s.n ? 'no trades yet' : s.n + ' trades, ' + Math.round
  const buy = Object.entries(ES.fam).filter(([, x]) => x.sign >= 0), warn = Objec
  const live = ((ER && ER.events) || []).filter(e => e.f.split(',').some(k => ES.
  return ['COMPANY NEWS -> WHAT THE STOCK DID NEXT (SEC filings and FDA approvals
    'AS A BUY (news + a strong day, bought the next open; must beat stocks that j
    '', 'WARNINGS (stocks lagged the market and other stocks by 1%+ over the next
    ...(live.length ? ['', 'LAST 10 DAYS (only kinds that passed or warn):', ...l
    '', 'SUPPLY CHAIN (hot industry -> its suppliers next month): ' + (ES.link ?
    'PEER LAG (industry leader jumps 6%+ -> its laggard peers for 3 days): ' + (E
    'Only kinds that PASSED become buys (as "event drift" in ACTIONS). Research,

// The rules come from the page itself, so the page and the scanner can never dis
const html = fs.readFileSync('index.html', 'utf8'), m = html.match(/<script id="l
if (!m) { console.error('index.html has no logic block. Upload the latest index.h
const tmp = path.join(os.tmpdir(), 'logic-' + Date.now() + '.cjs'); fs.writeFileS
const L = createRequire(import.meta.url)(tmp);

const readJson0 = (f) => { try { return JSON.parse(fs.readFileSync(f, 'utf8')); }
const DEFAULT = 'SPY QQQ IWM DIA AAPL MSFT NVDA AMZN GOOGL META TSLA AVGO AMD NFL
const fromFile = fs.existsSync('watchlist.txt') ? fs.readFileSync('watchlist.txt'
// Candidates from the discovery engine are tested automatically, so a Buy watch
let DISC = new Set(), BIOBLOCK = new Map(), DISCMETA = null;
try { DISCMETA = JSON.parse(fs.readFileSync('data/discovery.json', 'utf8')); cons
// YOUR HALAL MARKS from ZAD: "halal BFLY no" blocks it everywhere (plan, radar,
const HALAL_FILE = 'data/halal.json', HALAL_DAYS = 90;
// FILTERING POLICY: only YOUR OWN ZAD marks ever remove a stock. Every other scr
```

```
// HalalScreener) is SHOWN on the card as information, never used to drop a stock
const myHalal = (sym) => { const m = HALALM()[sym]; return m && (m.s === 'fail' |
// your ZAD marks. A NOT HALAL mark counts for 90 days, then expires: companies c
// stock can become halal again. When an expired one shows up as a buy, the card
// ZAD's published list (data/zad-list.json, from its quarterly Sharia review) fi
// only when it is newer than the list. Stocks the list removes are NOT HALAL rig
// on the date they stop being compliant).
const ZADL = () => { const z = readJson0('data/zad-list.json'); if (!z) return {}
  for (const s of z.notHalal || []) o[s] = { s: 'fail', d, by: 'ZAD list (not com
const HALALM = () => { const z = ZADL(), m0 = readJson0(HALAL_FILE) || {}, m = {
// ZOYA (AAOIFI) halal pre-check. Secret ZOYA_KEY holds the key WITH its prefix (
// the buy list is checked about weekly and remembered in data/zoya.json. The res
// from the buy list; your own ZAD mark always wins. Sandbox data are dummy value
const ZOYA_FILE = 'data/zoya.json', ZOYAM = () => readJson0(ZOYA_FILE) || {};
const zoyaLive = () => /^live-/.test(env.ZOYA_KEY || '');
async function zoyaCheck(syms) {
  if (!env.ZOYA_KEY) return 0;
  const url = local(env.ZOYA_BASE) ? local(env.ZOYA_BASE) + '/graphql' : (zoyaLiv
  for (const sym of [...new Set(syms)].slice(0, 40)) {
    const old = Z[sym]; if (old && Date.now() - Date.parse(old.at) < 7 * 86400000
    for (let a = 0; a < 2; a++) try { const r = await fetch(url, { method: 'POST'
      if (!r.ok) { console.log('Zoya: HTTP ' + r.status + ' for ' + sym); if (r.s
      const j = await r.json(), rep = j && j.data && j.data.basicCompliance ? j.d
      Z[sym] = { s: rep ? String(rep.status || 'UNRATED') : 'UNRATED', p: rep &&
    } catch (e) { console.log('Zoya: ' + e.message + (a ? '' : ', retrying')); aw
    await sleep(FAST ? 0 : 300);
  }
  fs.writeFileSync(ZOYA_FILE, JSON.stringify(Z, null, 1)); return n;
}
// MORE HALAL PRE-CHECKS: Halal Terminal (secret HALALTERMINAL_KEY, free key abou
// (secret HALALSCREENER_KEY). Same rules as Zoya: each stock at most once every
const HEXT_FILE = 'data/halal-ext.json', HEXTM = () => readJson0(HEXT_FILE) || {}
const verdictOf = (j) => {   // tolerant reading of a provider's answer: true = p
  if (!j || typeof j !== 'object') return null; const o = j.data && typeof j.data
  for (const k of ['aaoifi_compliant', 'is_compliant', 'compliant', 'isCompliant'
  const st = String(o.status || o.compliance || o.compliance_status || o.verdict
  if (/NOT|NON|FAIL|HARAM|NO_?PASS/.test(st)) return false; if (/^(COMPLIANT|HALA
async function halalExtCheck(syms) {
  const SRC = [];
  if (env.HALALTERMINAL_KEY) SRC.push({ k: 'ht', name: 'Halal Terminal', max: 10,
  if (env.HALALSCREENER_KEY) SRC.push({ k: 'hs', name: 'HalalScreener', max: 25,
  if (!SRC.length) return 0; const X = HEXTM(); let n = 0;
  for (const src of SRC) { let used = 0;
    for (const sym of [...new Set(syms)]) { if (used >= src.max) break;   // free
      const old = X[sym] && X[sym][src.k]; if (old && Date.now() - Date.parse(old
      try { const [u, o] = src.req(sym); let r = null; for (let a = 0; a < 2 && !
        if (r.status === 401 || r.status === 402 || r.status === 403 || r.status
```

```
        if (!r.ok) continue; const j = await r.json().catch(() => null), v = verd
        X[sym] = X[sym] || {}; X[sym][src.k] = { v, pur: j && typeof (j.purificat
      } catch (e) { console.log(src.name + ': ' + e.message); }
      await sleep(FAST ? 0 : 400); } }
  fs.writeFileSync(HEXT_FILE, JSON.stringify(X, null, 1)); return n;
}
// All halal pre-checks for one stock: { pass: [...names], fail: [...names], text
const halalChecks = (sym) => {
  const pass = [], fail = [], unk = [], z = ZOYAM()[sym], x = HEXTM()[sym] || {};
  if (z && z.live) (z.s === 'COMPLIANT' ? pass : z.s === 'NON_COMPLIANT' ? fail :
  for (const [k, name] of [['ht', 'Halal Terminal'], ['hs', 'HalalScreener']]) if
  const parts = [...pass.map(n => n + ' PASS'), ...fail.map(n => n + ' FAIL'), ..
  return { pass, fail, text: parts.length ? 'Halal pre-checks: ' + parts.join(' |
};
const zoyaText = (sym) => { const z = ZOYAM()[sym]; if (!z) return null; const ta
  return 'Halal pre-check (Zoya, AAOIFI): ' + (z.s === 'COMPLIANT' ? 'COMPLIANT'
const EXTRA_FILE = 'data/watch-extra.json', readExtra = () => { try { const a = J
// Stocks the live whole-market radar flagged today (only if that feed is availab
const RADAR = (() => { const r = readJson0('data/intraday-radar.json'); return (r
// Names from the smart-money job (insider buying, analysts turning positive, upg
const SMART = (() => { const r = readJson0('data/smart-money.json'); return ((r &
const MAIN = ['SPY', ...new Set([...(fromFile.length ? fromFile : DEFAULT), ...re
// Extra large, heavily traded companies scanned for QUICK TRADES only. Short bou
const QEXTRA = env.NO_QUICK_EXTRA ? [] : 'ABBV ABT ACN ADBE AMAT AMGN BKNG BMY CA
const FMOVERS = (() => { try { const f = JSON.parse(fs.readFileSync('data/fm-stat
// Whole-market leaders: the strongest liquid stocks found by the morning sweep o
const LEADU = (() => { const u = readJson0('data/leaders-universe.json'); return
// WIDER HUNT: the most traded strong-trend COMPANIES from the morning whole-mark
// QUICK TRADES only. More stocks tested = more chances that a proven setup fires
// own trades the rule never saw, with the same 30-trade and after-cost rules. Se
const QWIDE = (() => { const n = env.WIDE_N !== undefined ? +env.WIDE_N || 0 : 12
  return ((u && u.syms) || []).map(x => x.sym).filter(x => /^[A-Z]{1,5}$/.test(x)
const BTSET = new Set([...(fromFile.length ? fromFile : DEFAULT), ...QEXTRA]);
const QONLY = new Set([...QEXTRA, ...FMOVERS, ...LEADU, ...QWIDE]), LEADHELD = ((
  LIST = [...new Set([...MAIN, ...QEXTRA, ...FMOVERS, ...LEADU, ...LEADHELD, ...Q
for (const x of LEADHELD) if (!MAIN.includes(x)) QONLY.add(x);

const sleep = (ms) => new Promise(r => setTimeout(r, ms));
const readJson = (f, d) => { try { return JSON.parse(fs.readFileSync(f, 'utf8'));
const money = (x) => x.toFixed(2);
fs.mkdirSync('data', { recursive: true });

function ny(ms) {
  const o = {}; new Intl.DateTimeFormat('en-CA', { timeZone: 'America/New_York',
  return { d: o.year + '-' + o.month + '-' + o.day, mins: +o.hour * 60 + +o.minut
}
// DAY-OF-WEEK: which weekday the next buy would happen on (today before the clos
```

```
function buyDayNote() {
  const D = readJson0('data/dow-study.json'); if (!D || !D.rows) return null;
  let ms = Date.now(), n = ny(ms); if (n.weekend || n.mins >= 960) { do { ms += 8
  const wd = new Date(n.d + 'T12:00:00Z').getUTCDay() - 1, r = D.rows[wd]; if (!r
  if (/WORSE/.test(r.verdict)) return 'DAY CHECK: buys made on a ' + r.day + ' di
  if (/BETTER/.test(r.verdict)) return 'DAY CHECK: buys made on a ' + r.day + ' d
  return null; }
// WHEN IN THE DAY: tested choice between the last minutes of the signal day and
function timingNote() { const T = readJson0('data/timing-study.json'); if (!T ||
  const w = (x) => x === 'CLOSE' ? 'in the last 15 minutes before the close (11:4
  const pre = T.buy === 'CLOSE' || T.sell === 'CLOSE';   // acting near the close
  return 'WHEN: buy ' + (T.buy === 'CLOSE' ? 'at the open after the first 15 minu
function timingText() { const T = readJson0('data/timing-study.json'); if (!T ||
  const l = (h, k) => T[h][k].n ? pcs(T[h][k].avg) + ' (' + T[h][k].n + ')' : 'n/
  return ['WHEN TO BUY AND SELL (tested on the unseen trades of the rules the sys
    'BUY at the next open: ' + l('older', 'openIn') + ' | ' + l('newer', 'openIn'
    'SELL (rule exit) at the next open: ' + l('older', 'openOut') + ' | ' + l('ne
    '', timingNote(), '',
    'Fixed rules (not testable with daily prices, from how the market works):',
    '- PRE-MARKET and AFTER-HOURS: do not trade. Few buyers and sellers, wide spr
    '- FIRST 15 MINUTES after the open: the widest swings of the day. Wait, unles
    '- Always LIMIT orders, never market orders.',
    '- STOPS: sell as soon as one is hit, any time in the session. Do not wait fo
    'A choice changes only when one way is better by 0.2% a trade or more in BOTH
function shadowText() {
  const LS = readJson0('data/lessons.json') || {}, rs = Object.values(LS.rules ||
  const n = rs.reduce((a, r) => a + r.n, 0), avg = n ? rs.reduce((a, r) => a + r.
  const ld = (LD.done || []).filter(x => typeof x.pct === 'number'), la = ld.leng
  const rg = Object.values(RR.groups || {}).filter(g => g.n), rn = rg.reduce((a,
  return ['SHADOW RECORD: every system signal followed forward on paper, whether
    'Tested-rule buys (swing and quick): ' + (n ? n + ' finished, ' + Math.round(
    'LEADERS trend trades: ' + (ld.length ? ld.length + ' finished, average ' + p
    'Early-wave radar: ' + (rn ? rn + ' finished, average ' + pcs(ra) : 'none fin
    '', 'This is the honest forward test: about 60 finished trades per kind are n
function themesText() { const F = readJson0('data/leaders-study.json'), th = F &&
  if (!th || !th.now) return 'THEMES: the industry study runs every Sunday with t
  return ['WHERE THE MONEY IS GOING (strongest industries over 6 months, from the
    '', 'TEST: ' + th.verdict + '.', ...(() => { const ES = readJson0('data/event
      return ['', 'SUPPLY CHAIN NOW (hot industry -> suppliers that have not move
    'Information to explore with  check SYM . A stock becomes a buy only through
function dowText() {
  const D = readJson0('data/dow-study.json'); if (!D || !D.rows) return 'The day-
  return ['DAY OF THE WEEK: does the buy day matter? (unseen trades of the rules
// ---- ALPACA PAPER TRADING (fake money only). Every fully proven BUY becomes a
// its stop attached; each night the trade's own tested exit closes it. Real fill
// The address is fixed to Alpaca's PAPER server, so no real money can ever move.
const ALP = local(env.ALPACA_BASE) || 'https://paper-api.alpaca.markets', ALP_FIL
```

```
async function alp(method, path, body) {
  if (!env.ALPACA_KEY || !env.ALPACA_SECRET) return null;
  try { const r = await fetch(ALP + path, { method, headers: { 'APCA-API-KEY-ID':
    const t = await r.text(); let j = null; try { j = t ? JSON.parse(t) : {}; } c
  catch (e) { console.log('Alpaca paper: ' + e.message); return null; }
}
async function alpacaPaper(sig, loaded, session) {
  if (!env.ALPACA_KEY || !env.ALPACA_SECRET || !sig) return;
  const A = readJson(ALP_FILE, { plans: {}, pendingBuy: {}, pendingExit: {}, clos
  const acct = await alp('GET', '/v2/account'); if (!acct || acct.error) { consol
  if (!A.start) A.start = { d: today, equity: +acct.equity };
  const posL = await alp('GET', '/v2/positions'), held = {}; for (const p of Arra
  // 1. finished exits from the last run: record the real fill
  for (const [sym, x] of Object.entries(A.pendingExit)) { const o = await alp('GE
    A.closed.push({ sym, kind: x.kind, entry: x.entry, exit: ex, pct: +pct.toFixe
    else if (o && ['canceled', 'expired', 'rejected'].includes(o.status)) delete
  // 2. stops the broker filled by itself (position gone, no exit sent by us)
  for (const [sym, P] of Object.entries(A.plans)) if (!held[sym] && !A.pendingExi
    const os = await alp('GET', '/v2/orders?status=closed&limit=20&symbols=' + sy
    if (st) { const ex = +st.filled_avg_price, pct = ex / P.entry - 1; A.closed.p
    delete A.plans[sym]; }
  // 3. new fills become tracked paper positions with their own exit rule
  for (const [sym, b] of Object.entries(A.pendingBuy)) { if (held[sym] && !A.plan
  // 4. tonight's exits, by each trade's own tested rule (stops sit at the broker
  for (const [sym, P] of Object.entries(A.plans)) { const d = loaded[sym]; if (!d
    const c = d.candles, i = c.length - 1, ind = L.indicators(c, null), n = c.fil
    if (P.kind === 'quick') { const s5 = ind.sma5[i]; if (n >= 1 && (P.q === 'gap
    else if (P.kind === 'leader') { if (c[i].c < ind.e[50][i]) why = 'closed unde
    else { const vv = P.key ? L.VARIANTS.find(x => L.vkey(x) === P.key) : null, l
      if (vv && L.exitAt(ind, c, vv, i)) why = 'rule exit'; else if (lv && c[i].c
    if (why) { const oo = await alp('GET', '/v2/orders?status=open&symbols=' + sy
      const o = await alp('POST', '/v2/orders', { symbol: sym, qty: String(P.qty)
      if (o && o.id) A.pendingExit[sym] = { id: o.id, entry: P.entry, qty: P.qty,
  // 5. tonight's proven buys become paper orders for the next session
  const cands = planCands(sig, {}).filter(x => x.ok && !held[x.sym] && !A.plans[x
  for (const x of cands.slice(0, 10)) { const usd = amountFor(x, sig.market, fals
    const src = x.kind === 'quick trade' ? (sig.quick || []).find(q => q.sym ===
    const plan = x.kind === 'quick trade' ? { kind: 'quick', q: src && src.q, max
    if (!plan) continue;
    const o = await alp('POST', '/v2/orders', { symbol: x.sym, qty: String(qty),
    if (o && o.id) A.pendingBuy[x.sym] = { id: o.id, d: today, plan: { ...plan, k
  A.closed = A.closed.slice(-500); A.equity = { d: today, equity: +acct.equity };
  console.log('Alpaca paper: ' + Object.keys(held).length + ' positions, ' + Obje
}
async function paperText() {
  if (!env.ALPACA_KEY || !env.ALPACA_SECRET) return 'PAPER ACCOUNT: not connected
  { const t = await alp('GET', '/v2/account'); if (!t || t.error) return 'PAPER A
```

```
  const A = readJson(ALP_FILE, { closed: [], plans: {}, pendingBuy: {} }), acct =
  const w = c.filter(t => t.net > 0), avg = c.length ? c.reduce((a, t) => a + t.n
  return ['PAPER ACCOUNT (Alpaca, fake money; real fills and timing)', acct && !a
    'Finished trades: ' + (c.length ? c.length + ', ' + Math.round(w.length / c.l
    'Open: ' + ((Array.isArray(pos) && pos.length) ? pos.map(p => p.symbol + ' '
    '', 'Only fully proven buys are placed, with their stop attached; each closes
const barClosed = (d, now) => { const n = ny(now); return d < n.d || (d === n.d &
// US stock market holidays, worked out from the exchange's standing rules (New Y
// Good Friday, Memorial Day, Juneteenth, Independence Day, Labor Day, Thanksgivi
// Early closes and one-off closures are not covered.
const HOLS = {};
function usHolidays(y) {
  if (HOLS[y]) return HOLS[y];
  const d = (m, day) => new Date(Date.UTC(y, m, day)), isoD = (x) => x.toISOStrin
  const nth = (m, wd, k) => { const f = d(m, 1); return d(m, 1 + (wd - f.getUTCDa
  const obs = (x) => x.getUTCDay() === 6 ? new Date(+x - 86400000) : x.getUTCDay(
  const a = y % 19, b = Math.floor(y / 100), c = y % 100, dd = Math.floor(b / 4),
  const easter = d(Math.floor((h + l - 7 * m + 114) / 31) - 1, (h + l - 7 * m + 1
  const list = [ny1.getUTCDay() === 6 ? null : obs(ny1), nth(0, 1, 3), nth(1, 1,
  return (HOLS[y] = new Set(list.filter(Boolean).map(isoD)));
}
const isHoliday = (dateStr) => usHolidays(+dateStr.slice(0, 4)).has(dateStr);
const marketOpen = (now) => { const n = ny(now); return !n.weekend && !isHoliday(

let used = 0;
async function td(pathname) {   // rate-limited Twelve Data call with one retry o
  for (let attempt = 0; attempt < 3; attempt++) {
    if (used >= RATE) { console.log('rate limit pause'); await sleep(FAST ? 5 : 6
    used++;
    let j; try { const r = await fetch(TD + pathname + '&apikey=' + encodeURIComp
    if (j.code === 429) { await sleep(FAST ? 5 : 61000); used = 0; continue; }
    return j;
  }
  return { status: 'error', message: 'rate limit' };
}
// Live prices: Finnhub quote first (60 calls a minute, free), so the Twelve Data
// price histories (that budget is what limits how many stocks can be tested). Tw
let fhUsed = 0, fhStart = Date.now();
async function liveQ(sym) {
  if (env.FINNHUB_KEY) {
    if (Date.now() - fhStart > 60000) { fhStart = Date.now(); fhUsed = 0; }
    if (fhUsed >= 50) { await sleep(FAST ? 5 : Math.max(0, 61000 - (Date.now() -
    fhUsed++;
    try { const r = await fetch(FH + '/quote?symbol=' + encodeURIComponent(sym) +
  }
  return td('/price?symbol=' + encodeURIComponent(sym));
}
```

```
// ---- WHOLE MARKET (paid Massive plan: unlimited calls, 5+ years of daily price
// night with the same rules and the same proof gates as the core list. On the fr
// switches this off by itself and the scan continues as before.
let mvHost = 0, mv429 = 0;
async function mv(pathq) {
  for (let a = 0; a < 3; a++) {
    try { const r = await fetch(MV_HOSTS[mvHost] + pathq + (pathq.includes('?') ?
      if (r.status === 429) { mv429++; if (mv429 > 6) return null; await sleep(FA
      if (!r.ok) return null; return await r.json(); }
    catch (e) { if (mvHost + 1 < MV_HOSTS.length) mvHost++; else await sleep(FAST
  }
  return null;
}
async function wideUniverse(already) {
  if (!env.MASSIVE_KEY || env.WIDE_ALL === '0') return [];
  const co = readJson0('data/companies.json'), cs = co && co.syms && co.syms.leng
  if (!cs) { console.log('whole market: no company list yet (discovery makes it),
  const days = []; for (let k = 0; k < 10 && days.length < 5; k++) { const d = ne
    const j = await mv('/v2/aggs/grouped/locale/us/market/stocks/' + d.toISOStrin
    if (j && Array.isArray(j.results) && j.results.length > 500) days.push(new Ma
  if (days.length < 3) return [];
  const MIN_DV = +(env.WIDE_DV || 1e7), MAX = +(env.WIDE_MAX || 2500), out = [];
  for (const [sym, r] of days[0]) {
    if (!/^[A-Z]{1,5}$/.test(sym) || !cs.has(sym) || already.has(sym) || !(r.c >=
    const dv = days.map(m => m.get(sym)).filter(Boolean).map(x => x.c * x.v).sort
    const med = dv[Math.floor(dv.length / 2)]; if (med >= (r.c < 5 ? MIN_DV / 2 :
  }
  return out.sort((a, b) => b.dv - a.dv).slice(0, MAX).map(x => x.sym);
}
async function loadWide(syms, loaded, now) {
  const from = new Date(now - (365 * 5 + 10) * 86400000).toISOString().slice(0, 1
  let i = 0, ok = 0; const worker = async () => { while (i < syms.length && mv429
    const j = await mv('/v2/aggs/ticker/' + sym + '/range/1/day/' + from + '/' +
    if (!j || !Array.isArray(j.results) || j.results.length < 30) continue;
    const all = j.results.map(x => ({ d: new Date(x.t).toISOString().slice(0, 10)
    loaded[sym] = { candles: all.filter(c => barClosed(c.d, now)), last: all[all.
  await Promise.all(Array.from({ length: +(env.WIDE_CONC || 8) }, worker));
  return ok;
}
async function telegram(text, buttons) {   // buttons: optional rows of { text, d
  if (!env.TG_TOKEN || !env.TG_CHAT) { console.log('[no Telegram secrets, message
  const parts = []; { let rest = text; while (rest.length > 3800) { let cut = res
  for (let i = 0; i < parts.length; i++) {
    for (let a = 0; a < 3; a++) {   // a dropped connection is retried, so a mess
      try { const r = await fetch(TG + '/bot' + env.TG_TOKEN + '/sendMessage', {
        if (!r.ok) console.error('Telegram error', r.status, await r.text()); bre
      catch (e) { console.error('Telegram connection error, retrying', e.message)
```

```
    }
    if (i + 1 < parts.length) await sleep(FAST ? 0 : 700);   // keeps the parts i
  }
}
const FOOT = '\nTested signals, not instructions. Not financial advice, not a hal

// ---- Your own positions, told to the bot in Telegram. Read at every run (every
// Stored in data/my-positions.json: ticker, entry price, levels and date. No sha
const POS_FILE = 'data/my-positions.json';
// GUIDE TARGETS: a stock bought without a tested target (trend trades, swing tra
// two sell markers: 2x and 3x the risk to its stop (stop 8% under the buy -> +16
// PRICE TARGETS: only tested ones. A target is shown only when the walk-forward
// choice) found it beat the plain exit on trades the system never saw. Otherwise
// Earlier untested guide targets on your positions are removed here.
function guideTargets(P) { if (P && P.guide) { delete P.t1; delete P.t2; delete P
  if (P && P.kind === 'quick' && P.t1 && !P.ownT && (!P.tmode || !P.tnote)) { del
  return P; }
const loadPos = () => { const p = readJson(POS_FILE, {}); for (const k of Object.
const savePos = (p) => fs.writeFileSync(POS_FILE, JSON.stringify(p, null, 1));
const HELP = 'You do not need these. Tap the buttons in the bar (send  menu  to s
// ONE-TAP MENU. Telegram shows these in the "/" menu (bottom-left Menu button) a
// Every entry sends a normal command, so tapping and typing do the same thing.
const MENU = [
  ['actions', 'actions', 'The one message to act on: sell, buy (amount, stop), ho
  ['mystocks', 'my stocks', 'What to do with each stock you own'],
  ['day', 'market today', 'Is today a buying, selling or nothing-to-do day? (test
  ['today', 'today', 'Short plan with live prices'],
  ['check', 'check a stock', 'Deep check of any ticker: send /check NVDA'],
  ['bought', 'i bought', 'Record a buy: /bought NVDA 180.5 10 shares'],
  ['sold', 'i sold', 'Record a sale: /sold NVDA 190.2'],
  ['stopplaced', 'stop placed', 'You placed ALL stops at your broker'],
  ['market', 'market', 'Indexes, sectors and headlines now'],
  ['movers', 'movers', 'Stocks up 20%+ now, with the news (do not chase)'],
  ['daytrades', 'day trades', 'Fast movers: tested day-trade rules and today\u201
  ['newswatch', 'news watch', 'Threat headlines about your stocks and their compe
  ['radar', 'radar', 'Early-wave board (paper only)'],
  ['leaders', 'leaders', 'Strongest stocks to ride, with stop and exit'],
  ['themes', 'themes', 'Where the money goes: industries and supply chain'],
  ['events', 'events', 'Company news (SEC, FDA): what it did next, tested'],
  ['calendar', 'calendar', 'This week: CPI, jobs, Fed, holidays, your dividends']
  ['skips', 'skips', 'Buys you skipped, why, and what they did since'],
  ['health', 'health', 'Is every part of the system running?'],
  ['research', 'research', 'Every idea tested, result, and what is waiting'],
  ['review', 'review', 'Hold-or-sell study of every stock you own'],
  ['record', 'record', 'Your real results'],
  ['goals', 'goals', 'Money goals and timeline'],
  ['paper', 'paper', 'Alpaca PAPER account (fake money study)'],
```

```
  ['shadow', 'shadow', 'Forward paper record of every system signal'],
  ['timing', 'timing', 'Buy and sell at the open or the close?'],
  ['days', 'days', 'Does the buy day matter?'],
  ['lab', 'lab', 'This week's hunting lab results'],
  ['score', 'score', 'Every stock scored 0 to 100'],
  ['sources', 'sources', 'Scoreboard of the sources you logged'],
  ['details', 'details', 'Long version with every setup'],
  ['report', 'report', 'Last night's full scan report'],
  ['quieton', 'quiet on', 'Only action messages and alerts'],
  ['quietoff', 'quiet off', 'Every message'],
  ['help', 'help', 'All commands explained'],
  ['menu', 'menu', 'Show the button bar again'],
], MENU_V = 9;
const MENU_KB = [['actions', 'my stocks', 'market today'], ['check a stock', 'i b
const MENU_TEXT = ['MENU: tap a button in the bar under the typing box, or the  /
  'EVERY DAY: market today (buying, selling or nothing-to-do day) | actions (what
  'ANY STOCK: check a stock (deep check: tested rules, news, SEC filings, analyst
  'IDEAS AND STUDIES: leaders | themes | events | timing | days | lab | score', '
  'You can still type anything in plain words (Arabic or English), or forward a m
async function sendMenu() {
  if (!env.TG_TOKEN || !env.TG_CHAT) return;
  const post = (m, b) => fetch(TG + '/bot' + env.TG_TOKEN + '/' + m, { method: 'P
  await post('setMyCommands', { commands: MENU.map(([c, , d]) => ({ command: c, d
  await post('setChatMenuButton', { chat_id: env.TG_CHAT, menu_button: { type: 'c
  await post('sendMessage', { chat_id: env.TG_CHAT, text: MENU_TEXT, reply_markup
}
// commands that need a ticker or a price: tapped alone, show exactly what to sen
const ASK = { 'check a stock': 'Send:  check NVDA   (any ticker). I check the tes
  'i bought': 'Send:  bought NVDA 180.50 10 shares   (ticker, your price, shares)
  'i sold': 'Send:  sold NVDA 190.20   (all)  or  sold NVDA 190.20 5 shares   (pa
function planFor(sym, entry, sig) {   // levels for a new position, scaled from l
  const q = ((sig && sig.quick) || []).find(x => x.sym === sym), b = ((sig && sig
  if (q) return { kind: 'quick', q: q.q || 'rsi', key: q.key || null, maxHold: q.
  const ld = ((sig && sig.leaders) || []).find(x => x.sym === sym);
  if (!q && !b && ld) return { kind: 'leader', stop: +(entry - (ld.buyLo - ld.sto
  if (b) return { kind: 'entry', key: b.key || null, maxHold: b.maxHold || null,
  const ev = ((sig && sig.evt) || []).find(x => x.sym === sym && x.live); if (ev
  const pe = ((sig && sig.pead) || []).find(x => x.sym === sym && x.live); if (pe
  { const OD = readJson('data/octopus-daily.json', null), z = OD && OD.live && OD
  const rd = readJson('data/intraday-radar.json', null), rc = rd && (rd.candidate
  if (rc) return { kind: 'radar', maxHold: 5, stop: +(entry - (rc.levels.entry -
  return { kind: 'manual', stop: null, t1: null, t2: null };
}
const stopTag = (sym, P, short) => !P.stop ? (short ? ' NO STOP' : ' (NO STOP: se
const posLine = (sym, P, price) => sym + ': bought ' + money(P.entry) + (price ?
// "today": the short-trade list. Only setups that passed the tests at the last c
// ---- YOUR REAL RECORD: every sale you report, split by the kind of trade, so p
```

```
const MY_TRADES = 'data/my-trades.jsonl';
function myRecord(full) {
  const all = fs.existsSync(MY_TRADES) ? fs.readFileSync(MY_TRADES, 'utf8').split
  if (!all.length) return 'YOUR REAL RECORD: no sale recorded yet. When you sell,
  const line = (name, a) => { if (!a.length) return null; const w = a.filter(t =>
    return name + ': ' + a.length + ' trades, ' + Math.round(w.length / a.length
  const names = { quick: 'Quick trades', entry: 'Swing trades', leader: 'Leaders'
  const rows = [line('All system trades', all.filter(t => t.kind !== 'manual')),
  // CALIBRATION: did the system's buys do what their tests promised?
  const cal = all.filter(t => t.test && typeof t.test.win === 'number');
  if (cal.length) { const promW = cal.reduce((a, t) => a + t.test.win, 0) / cal.l
    rows.push('Promise vs reality (' + cal.length + ' system buys): tests said '
  if (cal.length >= 20) { const pw = cal.reduce((a, t) => a + t.test.win, 0) / ca
    if (Math.abs(dr) > 0.10) rows.push('CALIBRATION ALERT: the tests promised ' +
  { const ls = all.filter(t => t.lossType); if (ls.length) { const by = {}; for (
  return 'YOUR REAL RECORD (after about 0.4% costs; in dollars, your $' + 2 * FEE
}
// ---- MONEY GOALS (encrypted, like your share counts). Realized profit after fe
// (your TTD loss included). Ladder: back to zero, then 10K, 25K, 50K, 100K, 250K
// much the system tells you to risk: chasing a goal with bigger bets is the fast
const GOAL_LADDER = [0, 10000, 25000, 50000, 100000, 250000, 500000, 1000000];
const goalName = (g) => g === 0 ? 'Back to zero (losses recovered)' : money(g).re
function addRealized(pv, sym, usd, d, note) {
  if (typeof pv.realized !== 'number') pv.realized = (pv.closed || []).reduce((a,
  pv.closed = (pv.closed || []).concat({ sym, usd: +usd.toFixed(2), d, ...(note ?
  if (!pv.goals) pv.goals = { start: d, base: +(pv.realized - usd).toFixed(2), hi
}
function goalNews(pv) {
  if (!pv.goals) return ''; const prog = pv.realized - pv.goals.base, out = [];
  for (const g of GOAL_LADDER) if (prog >= g && !pv.goals.hit.includes(g) && (g >
  return out.length ? '\n\n' + out.join('\n') + ' Well done. Same rules, same siz
}
function goalsText() {
  const pv = privRead(), notYet = !pv.goals; if (notYet) { pv.realized = typeof p
  const prog = pv.realized - pv.goals.base, since = (pv.closed || []).filter(x =>
  const days = Math.max(1, (Date.now() - Date.parse(pv.goals.start)) / 86400000),
  const w = since.filter(x => x.usd > 0), l = since.filter(x => x.usd <= 0);
  const lines = [notYet ? 'MONEY GOALS (they start with your next recorded sale,
  for (const g of GOAL_LADDER) { const done = pv.goals.hit.includes(g); lines.pus
  lines.push(since.length >= 5 && days >= 30 && perMonth > 0 && next !== undefine
  // TIMELINE PLAN: your buy size x the expected profit a trade x trades a month
  { const sg0 = readJson('data/signals.json', null), okc = sg0 ? planCands(sg0, {
    const tpm = pv.tradesPerMonth || (buys30 >= 5 ? buys30 : 10);
    let net = null, src = '';
    if (mine.length >= 10) { net = mine.reduce((a, x) => a + x.usd, 0) / mine.len
    else { const sg = readJson('data/signals.json', null), ok = sg ? planCands(sg
    if (net === null) { lines.push('', 'TIMELINE PLAN: NO CURRENT PROVEN SETUP, s
```

```
    else {
    const at = (sz) => (net + 2 * FEE() / STD_BUY - 2 * FEE() / sz) * sz, month =
    const left = GOAL_LADDER.filter(g => !pv.goals.hit.includes(g)).slice(0, 4);
    const fmtT = (m) => !(m > 0) || !isFinite(m) ? 'not reachable at this pace' :
    lines.push('', 'TIMELINE PLAN: ' + money(size) + ' USD a buy' + (ub && ub.set
      'That is about ' + money(at(size)) + ' USD a trade and ' + money(month(size
    for (const g of left) lines.push('  ' + goalName(g) + ': ' + fmtT((g - prog)
    lines.push('Bigger buys reach goals faster but lose faster too: raise your bu
  lines.push('The goals never raise your size or risk. To add a sale I could not
  return lines.join('\n');
}
function lessonsText() {
  const LS = readJson('data/lessons.json', null), rs = LS ? Object.entries(LS.rul
  if (!rs.length) return 'LESSONS: no live paper trade has finished yet. Each fin
  const endName = { stop: 'stop', target: 'target', rule_exit: 'rule exit', time_
  return ['LESSONS FROM FINISHED TRADES (live paper, after costs), ' + LS.session
    + '\n  how they ended: ' + Object.entries(r.ends).map(([k, v]) => v + ' ' + (
    + (r.t1Rate !== null ? '\n  reached target 1: ' + Math.round(r.t1Rate * 100)
    + (r.fill && r.fill.inN + r.fill.upN ? '\n  opened inside the buy range ' + r
    + (r.demoted ? '\n  lesson: it loses live, so it is paper only until its reco
    'A rule is demoted automatically after 10+ live trades with a negative averag
}
function soldLesson(P, exit) {
  if (P.t2 && exit >= P.t2) return 'It reached target 2: the plan worked in full.
  if (P.t1 && exit >= P.t1) return 'It reached target 1: the plan worked.';
  if (P.stop && exit <= P.stop * 1.002) return 'It hit the stop: the loss was cap
  if (P.kind === 'quick') return 'It was sold on the quick-trade rule (the bounce
  if (P.kind === 'radar') return 'It ended on the 5 day time limit before marker
  return 'Sold before the target and above the stop.';
}
// ---- THE PLAN: the one short message. At most 2 buys, each with buy range, sto
// A buy needs proof on trades it never saw: 30 or more, and a positive average a
// The tests already charged 0.1%, so 0.3% more is taken off here. No two buys fr
const MY_COST = 0.004, TEST_COST = 0.001, CHEAP_COST = 0.012;
// after-cost average for YOU: stocks under 5 USD are charged 1.2% a round trip (
// YOUR BROKER FEE: a fixed fee per buy and per sell (default 3 USD each, change
// 1,000 USD buy: fixed fees + spread (0.2%, or 1% under 5 USD) must still leave
const FEE = () => { const st = readJson0('data/settings.json') || {}; return type
const costPct = (px, usd) => 2 * FEE() / (usd || STD_BUY) + (px > 0 && px < 5 ? 0
const netOf = (avg, px) => typeof avg === 'number' ? avg + TEST_COST - costPct(px
// always shown per $1,000, so every card compares the same way
const usdLine = (net, gross, px) => { if (net === null) return ''; const sz = STD
  return ' (on a ' + '$' + Math.round(sz) + ' buy: about ' + (n2 >= 0 ? '+' : '-'
// Ranking by evidence, not by luck: average after costs, weighted by how many un
const evid = (net, n) => (net || 0) * Math.sqrt(Math.min(n || 0, 200));
const secGroup = (x) => !x ? null : /biotech|pharma|life sciences|health/i.test(x
const pcs = (x) => (x >= 0 ? '+' : '') + (x * 100).toFixed(1) + '%';
```

```
// BUY evidence: 30+ unseen trades with a positive average after costs, OR an EAR
// with a strong average (+1.5% or more after costs) and at least half won. Early
// demoted automatically if their live record turns negative.
// MIN_NET: a proven buy must also clear +0.4% a trade after your $6 fees and the
const MIN_NET = 0.004;
const okEv = (n, net, win) => (n >= 30 && net >= MIN_NET) || (n >= 15 && net >= 0
// The full picture of a tested rule on this stock, after your fees and spread: a
// SAFETY MARGIN (lower bound: the average minus 1.28 standard errors, so there i
// lower). RANK = that lower bound divided by what the stop would cost. It ranks
const distOf = (pcts, px) => { if (!Array.isArray(pcts) || pcts.length < 5) retur
  const w = r.filter(x => x > 0), l = r.filter(x => x <= 0), sd = Math.sqrt(r.red
  return { n, avg: m, aw: w.length ? gw / w.length : 0, al: l.length ? -gl / l.le
// HIGH CONVICTION label (a label only, nothing is hidden): 60+ unseen trades, +0
// bound on the win rate of at least 50%, stop no wider than 12%, and not losing
const wilsonLo = (p, n) => { if (!(n > 0) || typeof p !== 'number') return 0; con
const highConv = (x) => !x.thin && x.n >= 60 && x.net >= 0.008 && wilsonLo(x.win,
const rankOf = (x) => { if (!x.d || !(x.lo > 0) || !(x.stop > 0)) return null; co
const distText = (d) => !d ? '' : 'Winners averaged ' + pcs(d.aw) + ', losers ' +
function planCands(sig, held0) {
  const c = [], HM = HALALM(), held = new Proxy(held0 || {}, { get: (o, k) => o[k
  for (const q of sig.quickPaused ? [] : (sig.quick || [])) { if (held[q.sym]) co
    c.push({ sym: q.sym, kind: 'quick trade', n: q.n, net, live: q.live, lo: q.bu
      sell: (q.t1 ? money(q.t1) + (q.half ? ' (half, tested; the rest on ' : ' (a
      wait: 'up to ' + (q.maxHold || 5) + ' trading days, then sell anyway (holdi
      expect: (q.n ? Math.round(q.win * 100) + '% of ' + q.n + ' past trades it n
      why: q.q === 'gap' ? 'strong gap up on heavy volume, held near the high' :
      ok: okEv(q.n, net, q.win) && !q.demoted, thin: q.n < 30, cheap: q.last < 5,
  for (const b of (sig.stocks || []).filter(x => x.state === 'Buy watch' && x.buy
    c.push({ sym: b.sym, kind: 'swing trade', n: b.n, net, live: b.live, lo: b.bu
      sell: b.target ? money(b.target) + (b.half ? ' (half, tested; the rest on a
      wait: (b.days ? 'about ' + b.days + ' trading days is typical' : 'days to w
      expect: (b.n ? Math.round(b.win * 100) + '% of ' + b.n + ' past trades it n
      why: String(b.why || 'a tested entry signal fired').split('. ')[0].replace(
      ok: okEv(b.n, net, b.win) && !b.demoted, thin: b.n < 30, cheap: b.last < 5,
  const RD = readJson('data/intraday-radar.json', null);
  if (RD && RD.session === ny(Date.now()).d && marketOpen(Date.now())) for (const
    const L0 = r.levels; c.push({ sym: r.sym, kind: 'early-wave ' + (String(r.gro
      sell: money(L0.t1) + ' (marker 1), or after 5 trading days', wait: 'up to 5
      why: (String(r.group || '').startsWith('pullback') ? 'broke the first-45-mi
      ok: true, score: 0.0005 }); }
  const fair = (sig.leadFair && sig.leadFair.state) || 'NOT RUN';
  for (const r of sig.leaders || []) { if (held[r.sym]) continue;
    c.push({ sym: r.sym, kind: 'trend trade', lo: r.buyLo, hi: r.buyHi, stop: r.s
      sell: 'no fixed target. Sell when it closes under the stop (I raise it as i
      wait: 'weeks, as long as the trend holds',
      expect: fair === 'PROVISIONALLY PROMISING' ? 'passed the whole-market fair
      why: 'one of the strongest stocks: ' + pcs(r.mom) + ' in 6 months',
```

```
      ok: fair === 'PROVISIONALLY PROMISING', score: -0.001 }); }
  for (const e of (sig.pead || []).filter(x => x.live && !held[x.sym])) { const t
    c.push({ sym: e.sym, kind: 'earnings drift', n: t.n || null, net: typeof t.av
      sell: 'when it closes under its 20 day average (~' + money(e.sma20) + ' now
      expect: (t.n ? Math.round((t.win || 0) * 100) + '% of ' + t.n + ' past earn
      why: 'earnings beat the estimate by ' + Math.round(e.sur * 100) + '% and th
  for (const e of (sig.evt || []).filter(x => x.live && !held[x.sym] && !(sig.pea
    c.push({ sym: e.sym, kind: 'event drift', n: t.n || null, net: typeof t.avg =
      sell: 'when it closes under its 20 day average (~' + money(e.sma20) + ' now
      expect: (t.n ? Math.round((t.win || 0) * 100) + '% of ' + t.n + ' past case
      why: e.name + ' (filed ' + e.p + ' New York time), and the stock closed up
  // OCTOPUS: the many-angle model's top 5 of the whole market, ONLY when its 5-y
  // passed (30+ picks over 10+ days, positive, half or more won). Exit: the clos
  { const OD = readJson0('data/octopus-daily.json'); if (OD && OD.live && OD.toda
      c.push({ sym: z.sym, kind: 'octopus', n: t ? t.n : null, net: t ? t.avg : n
        sell: 'at the close on the 10th trading day after you buy (the tested exi
        expect: (t ? Math.round((t.win || 0) * 100) + '% of ' + t.n + ' unseen 5-
        why: 'top 5 of the whole market on 26 linked angles (momentum, volume, in
  // NEWS GATE (precaution, not tested): no buy today for a stock the news watch
  { let L = []; try { L = fs.readFileSync('data/news-log.jsonl', 'utf8').trim().s
    const bad = new Set(L.filter(a => a.th && a.sym && Date.now() - Date.parse(a.
  // MACRO GATE (tested, calendar study): no new buy in an industry that lagged t
  // a buy in an industry that beat the market is marked as a tailwind and moved
  for (const m of macroNow()) for (const x of c) { const g = m.indOf(x.sym); if (
    if (x.ok && m.downSet.has(g)) { x.ok = false; x.macroBlock = m.label; } else
  c.sort((a, b) => (b.macroUp ? 1 : 0) - (a.macroUp ? 1 : 0));
  // EARLY EVIDENCE IS WATCH-ONLY (owner's decision, 12 Oct): 15-29 unseen trades
  // listed in one WATCH line; they become BUYs by themselves once they reach 30+
  for (const x of c) if (x.thin && x.ok) { x.ok = false; x.watch = true; }
  // BAD-NEWS GATE: no buy within 10 days of a filing kind the event study proved
  for (const x of c) if (x.ok && eventWarn(x.sym)) { x.ok = false; x.newsBlock =
  // FALLING-MARKET GATE: when the S&P 500 is under its 50 day average or in a do
  // its OWN unseen trades made money in falling markets (10+ such trades). Its r
  { const mk = sig.market || {}, weak = mk.above50 === false || mk.up === false;
    if (weak) for (const x of c) if (x.ok && /quick|swing/.test(x.kind || '')) {
      if (!dn || dn.n < 10 || !(dn.avgR > 0)) { x.ok = false; x.regBlock = true;
  return c;
}
// Your accepted loss per trade (set with:  risk 50). Only this one number is sto
const myRisk = () => null;     // retired: the system sizes each buy itself
// PRIVATE FIGURES (share counts, money amounts). The repository is public, so th
// the GitHub secret PRIVATE_KEY. Without that secret nothing is stored: the numb
const PRIV_FILE = 'data/private.enc', privKey = () => env.PRIVATE_KEY ? crypto.cr
function privRead() {
  const k = privKey(); if (!k || !fs.existsSync(PRIV_FILE)) return { buys: [], sh
  try { const b = Buffer.from(fs.readFileSync(PRIV_FILE, 'utf8'), 'base64'), d =
  catch (e) { return { buys: [], shares: {} }; }
```

```
}
function privWrite(o) {
  const k = privKey(); if (!k) return false; const iv = crypto.randomBytes(12), c
  fs.writeFileSync(PRIV_FILE, Buffer.concat([iv, c.getAuthTag(), enc]).toString('
}
// Your usual buy, learned from what you actually buy (the median of your last 10
const usualBuy = () => { const pv0 = privRead(); if (pv0.buyUSD > 0) return { usd
// The account size is private: it comes ONLY from the GitHub secret ACCOUNT_USD,
const myAccount = () => { const v = +(env.ACCOUNT_USD || 0); return v > 0 ? v : n
{ const st = readJson('data/settings.json', null); if (st && (st.accountUSD !== u
// SIZING DECIDED BY THE SYSTEM. How much of the account a trade may lose depends
// done live, and the market. 0.5% is the normal bet; strong, long records earn u
// market or a paper-only setup get less. A position is never more than a fifth o
function sizeFor(x, mk) {
  const n = x.n || 0, net = x.net === undefined || x.net === null ? null : x.net,
  let pct = 0.005;
  if (x.kind && /radar/.test(x.kind)) { pct = 0.0025; why.push('early-wave setup:
  else if (x.thin) { pct = 0.0025; why.push('EARLY EVIDENCE, only ' + n + ' teste
  else if (n >= 100 && net >= 0.015) { pct = 0.01; why.push('long, strong record
  else if (n >= 60 && net >= 0.01) { pct = 0.0075; why.push('strong record (' + n
  else if (n < 40 || (net !== null && net < 0.005)) { pct = 0.0035; why.push('thi
  else why.push('normal evidence (' + n + ' trades' + (net !== null ? ', ' + pcs(
  if (x.cheap) { pct *= 0.5; why.push('cheap stock under 5 USD: halved'); }
  if (x.live && x.live.n >= 10) { if (x.live.avg > 0) { pct *= 1.2; why.push('mak
  if (mk && (mk.up === false || mk.above50 === false)) { pct *= 0.5; why.push('we
  { const sd = x.lo > 0 && x.stop > 0 ? (x.lo - x.stop) / x.lo : 0; if (sd > 0.08
  return { pct: Math.min(0.01, pct), why: why.join(', ') };
}
// "Take half, let the rest run": used only when the Sunday exit test (hunting la
const HALF_RUN = () => { const h = readJson0('data/hunting-lab.json'); return Boo
const t1Text = (sym, P) => P.tmode === 'all' ? 'TARGET (' + money(P.t1) + ') reac
// HOW MUCH TO PUT IN EACH BUY: one of 1,000 / 1,500 / 2,000 / 3,000 / 5,000 USD,
// Never under 1,000 (the 6 USD fees eat a smaller buy). Your budget (send  budge
function amountFor(x, mk, top) {
  const weak = mk && (mk.up === false || mk.above50 === false), stopPct = x.lo >
  let usd = 1000, why = 'early evidence, cheap stock, trend trade or radar: the m
  if (!x.thin && x.n >= 30 && !x.cheap && /quick|swing|octopus|event drift|earnin
    if (hc && top && stopPct <= 0.08 && !weak) { usd = 5000; why = 'HIGH CONVICTI
    else if (hc) { usd = 3000; why = 'HIGH CONVICTION'; }
    else if (lbOk && stopPct <= 0.12) { usd = 2000; why = 'proven with a safety m
    else { usd = 1500; why = 'proven, but the edge could still be luck or the sto
  }
  if (weak && usd > 1000) { usd = usd >= 3000 ? usd / 2 : 1000; why += '; weak ma
  if (x.earnDays !== null && x.earnDays !== undefined && x.earnDays <= (x.days ||
  { const b = budgetOnce(); if (b && usd > Math.max(1000, Math.floor(b * 0.2 / 50
  // RISK PER TRADE: a hit stop should cost at most 1.5% of your budget (so 5 los
  // minimum (your choice); when even 1,000 USD risks more than that, the card sa
```

```
  let riskPct = null; { const b = budgetOnce(); if (b && stopPct > 0 && stopPct <
    if (usd > cap) { usd = Math.max(1000, cap); why += cap >= 1000 ? '; kept so a
  return { usd, why, riskPct };
}
let BUDGET_C; const budgetOnce = () => { if (BUDGET_C === undefined) { try { BUDG
const budgetRoom = () => { const pv = privRead(), b = pv.budget || 0; if (!b) ret
  // money that today's SELL lines free up (at the buy prices), so a full budget
  const selling = Object.keys(pos).filter(k => (pos[k].lastAct && /^SELL ALL/.tes
  const freed = selling.reduce((a, k) => a + ((pv.shares || {})[k] || 0) * (pos[k
  return { budget: b, open, room: Math.max(0, b - open), freed, after: Math.max(0
function planText(sig, pos, px, title, only, full, opts) {
  const short = !only && !full;   // the everyday message: 2 to 3 lines per buy;
  const held = pos || {}, out = [title || 'PLAN'];
  if (!sig) return out.concat('No scan has run yet.').join('\n');
  const c = planCands(sig, held), pick = c.filter(x => x.ok && (!only || only.inc
  for (const x of pick) { const g = secGroup(x.sector), first = g && pick.find(p
  const pv = sig.posVerdict || {}, sells = [], holds = [];
  for (const sym of Object.keys(held)) { const P = held[sym], p = px && px[sym],
    if (p && P.stop && p <= P.stop) sells.push(sym + ': SELL NOW. Your stop ' + m
    else if (p && P.t2 && p >= P.t2) sells.push(sym + ': SELL. Target 2 (' + mone
    else if (pv[sym]) sells.push(sym + ': SELL at the ' + (px ? 'close or next op
    else if (p && P.t1 && p >= P.t1 && !P.trail) sells.push(sym + ': ' + t1Text(s
    else holds.push(sym + ': stop ' + (P.stop ? money(P.stop) : 'NOT SET') + (P.t
  if (sells.length && !only) out.push('', 'SELL', ...sells);
  const mk = sig.market || {};
  if (!only && (mk.above50 === false || mk.up === false)) out.push('', 'MARKET: '
  for (const x of c) x.rank = rankOf(x);
  const RK = (x) => typeof x.rank === 'number' ? x.rank : -1e9;
  pick.sort((a, b) => (a.thin ? 1 : 0) - (b.thin ? 1 : 0) || RK(b) - RK(a) || b.s
  const nFull = pick.filter(x => !x.thin).length;
  // TOP PICK: the single best trade for its risk. It needs a positive safety mar
  // The full list stays below: how many you buy is your decision.
  let TOPSYM = null;
  if (!only && pick.length) { const hOK = (x) => { const hm = HALALM()[x.sym]; if
    const top = pick.filter(x => typeof x.rank === 'number' && x.d && x.d.lb > 0
    if (top) { TOPSYM = top.sym; const mid = (top.lo + top.hi) / 2, loss = Math.r
      out.push('', 'TOP PICK: ' + top.sym + (highConv(top) ? ' (HIGH CONVICTION)'
    else if (pick.some(x => x.d)) out.push('', 'TOP PICK: none today. No buy has
  if (!only) out.push('', short ? 'BUY (' + nFull + ' proven' + (pick.length > nF
  if (!only && !short && pick.length && !nFull) out.push('None fully proven (30+
  { const b = budgetOnce(); if (b && pick.length && !only) { const per = {}; for
      const over = Object.entries(per).filter(([g, v]) => g !== 'Other' && v > b
  { const br = budgetRoom(), m0 = (x) => money(x).replace(/\.00$/, ''); if (br &&
      if (br.room >= 1000) out.push('BUDGET: ' + m0(br.open) + ' USD open of ' +
      else if (br.after >= 1000) out.push('BUDGET FULL: ' + m0(br.open) + ' USD o
      else out.push('NO NEW BUYS TODAY. BUDGET FULL: ' + m0(br.open) + ' USD open
  if (!pick.length && !only) out.push(short ? 'Nothing proven today. Waiting is a
```

```
pick.forEach((x, n) => {
  if (short) {   // compact card
    if (x.thin && (n === 0 || !pick[n - 1].thin)) out.push('', 'EARLY EVIDENCE
    const p = px && px[x.sym], ex = String(x.expect || ''), wm = ex.match(/^(\d
    const sell = String(x.sell || '').replace(/ \(no fixed target\)/, '').repla
    const hm = HALALM()[x.sym], hc = halalChecks(x.sym), bi = L.halalOf(x.sym,
    const hal = hm && hm.s === 'ok' ? (/list/.test(hm.by || '') ? 'ZAD halal (Z
    const z = sizeFor(x, sig.market), stopPct = Math.round((1 - x.stop / x.lo)
    out.push((n ? '' : '') + (n + 1) + ') ' + x.sym + (highConv(x) ? ' HIGH CON
      '   Buy ' + money(x.lo) + '-' + money(x.hi) + ' | Stop ' + money(x.stop)
      '   ' + (typeof x.net === 'number' ? (x.net >= 0 ? '+' : '-') + '$' + Mat
      + (x.earnDays !== null && x.earnDays !== undefined && x.earnDays <= x.day
    return;
  }
  if (!only && x.thin && (n === 0 || !pick[n - 1].thin)) out.push('', 'EARLY EV
  const p = px && px[x.sym], risk = x.lo - x.stop;
  { const SS = readJson('data/score-study.json', null), sc = SS && SS.scores &&
    out.push((n ? '\n' : '') + (n + 1) + ') ' + x.sym + (highConv(x) ? ' (HIGH
  if (p) out.push('   Now: ' + money(p) + (p > x.hi ? ', ABOVE the range: SKIP,
  out.push('   Buy: ' + money(x.lo) + ' to ' + money(x.hi) + ' (skip if above '
    '   Stop: ' + money(x.stop) + ' (' + pcs(x.stop / x.lo - 1) + '). If it is
    '   Sell: ' + x.sell + (x.tgtNote ? '. Target study: ' + x.tgtNote : ''),
    '   Wait: ' + x.wait,
    '   Expected: ' + x.expect + (x.d ? '. ' + distText(x.d) : '') + (typeof x.
    '   Why: ' + x.why,
    ...(() => { const hm = HALALM()[x.sym], zt = zoyaLive() ? zoyaText(x.sym) :
    '   Amount: BUY ' + money(amountFor(x, sig.market, TOPSYM === x.sym).usd).r
    '   Size: ' + (!myRisk() ? (() => { const z = sizeFor(x, sig.market), acct
      if (!(risk > 0)) return 'n/a';
      if (!acct && ub) { const mult = z.pct / 0.005, usd = ub.usd * mult, sh =
        return sh < 1 ? 'skip: one share costs more than this trade should take
      if (!acct) return 'size ' + (z.pct / 0.005).toFixed(1) + 'x your usual bu
      const usd = acct * z.pct, sh = Math.min(Math.floor(usd / risk), Math.floo
      return sh < 1 ? 'skip: even 1 share risks ' + money(risk) + ' USD, more t
  if (x.sameAs) out.push('   NOTE: same industry as ' + x.sameAs + '. Both can
  if (x.earnDays !== null && x.earnDays !== undefined && x.earnDays <= x.days *
});
if (only) { out.push('', 'Check halal in ZAD first. Buy only inside the range a
if (short && opts && opts.actions) return out.slice(1).join('\n');   // the ACT
if (short) {
  if (holds.length) out.push('', 'HOLD (stops at your broker)', ...holds.map(h
  const RR = readJson('data/radar-record.json', null), rv = RR ? Object.values(
  out.push('', 'Tested ' + (sig.tested || sig.scanned || 0) + ' stocks | radar
  return out.join('\n');
}
const notProven = c.filter(x => !x.ok).map(x => x.sym);
if (skipped.length) out.push('', 'Also passed, left out: ' + skipped.join(', ')
```

```
  if (notProven.length) out.push('', 'Not proven, paper only: ' + [...new Set(not
  if (holds.length) out.push('', 'HOLD (keep these stops at your broker)', ...hol
  { const LS = readJson('data/lessons.json', null), rs = LS ? Object.values(LS.ru
    if (n) { const dem = rs.filter(r => r.demoted), sum = rs.reduce((a, r) => a +
      out.push('', 'LEARNING: ' + n + ' live paper trades reviewed, average ' + p
  { const RR = readJson('data/radar-record.json', null), gs = RR ? Object.values(
    out.push('', 'EARLY-WAVE RADAR (alerts during the session): ' + (pv.length ?
      : 'not proven yet' + (best && best.n ? ' (furthest: ' + best.name.toLowerCa
  { const ru = readJson('data/radar-universe.json', null), rn = ru && ru.syms ? r
    out.push('', 'COVERAGE: ' + (sig.tested || sig.scanned || 0) + ' stocks teste
  out.push('', 'Before buying: check halal in ZAD. Buy only inside the range and
  return out.join('\n');
}
// CONCLUSION: one line per trade. Buy range, stop, where to sell. Only setups th
function conclusion(sig, pos) {
  const out = ['CONCLUSION'];
  if (!sig) return out.concat('No scan has run yet.');
  const held = pos || {}, OK = new Set(planCands(sig, held).filter(x => x.ok).map
  for (const q of (sig.quick || []).filter(x => !held[x.sym] && OK.has(x.sym)).sl
  for (const b of (sig.stocks || []).filter(x => x.state === 'Buy watch' && x.buy
  for (const r of (sig.leaders || []).filter(x => !held[x.sym] && OK.has(x.sym)).
  if (sig.quickPaused) out.push('Quick trades are paused by the circuit breaker.'
  if (out.length === 1) out.push('No new trade today. Nothing passed its tests.')
  const pv = sig.posVerdict || {};
  for (const sym of Object.keys(pos || {})) { const P = pos[sym]; out.push(sym +
  return out;
}
// The same conclusion, recalculated from the price RIGHT NOW (market hours only)
// selling prices moved by the same amount. A setup that has run away or failed i
function conclusionLive(sig, pos, px) {
  const out = ['WHAT TO DO NOW (' + new Date().toLocaleTimeString('en-GB', { time
  for (const q of (sig.quick || []).filter(x => !held[x.sym] && OK.has(x.sym)).sl
    out.push(p > q.buyHi ? q.sym + ': SKIP. Now ' + money(p) + ', above its entry
      : q.sym + ': BUY near ' + money(p) + ' | STOP ' + money(p - (q.buyLo - q.st
  for (const b of (sig.stocks || []).filter(x => x.state === 'Buy watch' && x.buy
    out.push(p > b.buyHi ? b.sym + ': SKIP. Now ' + money(p) + ', above its buy r
      : b.sym + ': BUY near ' + money(p) + ' | STOP ' + money(p - (b.buyLo - b.st
  if (sig.quickPaused) out.push('Quick trades are paused by the circuit breaker.'
  if (!any) out.push('No new trade right now. Nothing passed its tests at the las
  const pv = sig.posVerdict || {};
  for (const sym of Object.keys(held)) { const P = held[sym], p = px[sym], chg =
    out.push(sym + ' (yours): ' + (p && P.stop && p <= P.stop ? 'SELL NOW, your s
  return out;
}
// WHAT KIND OF DAY IS TODAY? The market's state, each part checked against the m
// grouped by what the S&P 500 did), plus YOUR tested sells and the proven buys.
async function dayText(sig, pos) {
```

```
  const MD = readJson0('data/mday-study.json'), row = (k) => MD && MD.rows ? MD.r
  const sp = await loadDaily('SPY'), c = sp && sp.candles, n = c ? c.length : 0,
  if (n > 25) { const b = c[n - 1], r = b.c / c[n - 2].c - 1; tags.push(r <= -0.0
    let s1 = 0, s2 = 0; for (let j = n - 20; j < n; j++) { const x = c[j].c / c[j
    tags.push(sig && sig.market && sig.market.up === false ? 'weak' : 'strong');
    out.push('S&P 500 (SPY) last close ' + b.d + ': ' + pcs(r) + ' (' + NAME[tags
  // the buy morning: how the S&P 500 opened (known from 4:30 pm Kuwait)
  { const t = ny(Date.now()); if (!t.weekend && t.mins >= 570 && t.mins < 960 &&
      if (j && +j.o > 0 && +j.pc > 0) { const g = j.o / j.pc - 1; tags.push(g <=
    else out.push('The market is closed now (opens 4:30 pm Kuwait). The opening g
  // what the tests say about days like this
  const bad = [], good = [], lines = [];
  for (const k of tags) { const r = row(k); if (!r) continue; lines.push('- ' + N
    if (/LOSING|WORSE/.test(r.verdict)) bad.push({ k, r }); if (/BETTER/.test(r.v
  // your actions: tested sells, and the proven buys
  let h = { acts: { sell: [], part: [] } }; try { h = await holdingsText(sig, pos
  const sells = [...h.acts.sell, ...h.acts.part].map(l => l.split(/[ :(]/)[0]), b
  const losing = bad.some(x => /LOSING/.test(x.r.verdict)), head = [];
  if (sells.length) head.push('SELL DAY: ' + sells.length + ' of your stocks hit
  if (!buys.length) head.push('NO NEW BUYS: no proven buy rule fired.');
  else if (losing) head.push('NOT A BUYING DAY: buys made after days like this LO
  else if (bad.length) head.push('CAREFUL BUYING DAY: buys after ' + bad.map(x =>
  else head.push('BUYING DAY: ' + buys.length + ' proven buy' + (buys.length > 1
  if (!sells.length && !buys.length) head.splice(0, head.length, 'NOTHING TO DO T
  const dn = buyDayNote();
  return ['TODAY'S MARKET: ' + head.map(x => x.split(':')[0]).join(' + '), ...hea
    MD && MD.rows ? 'TESTED (the system's own unseen trades bought after days lik
    ...(dn ? ['', dn] : []), '', 'A day counts as worse or better only with 150+
}
async function hotList(sig) {
  if (!sig) return 'No scan has run yet.';
  const open = marketOpen(Date.now()), pos = loadPos(), px = {};
  if (open) { const c = planCands(sig, pos); for (const sym of [...new Set([...c.
  if (QUIET()) return actionsText(sig, pos, open ? px : null, open ? 'ACTIONS NOW
  return planText(sig, pos, open ? px : null, open ? 'WHAT TO DO NOW (' + new Dat
}
async function detailList(sig) {
  if (!sig) return 'No scan has run yet.';
  const open = marketOpen(Date.now()), pos = loadPos(), px = {};
  const OK = new Set(planCands(sig, pos).filter(x => x.ok).map(x => x.sym)), qk =
  if (open) for (const sym of [...new Set([...qk.map(q => q.sym), ...buys.map(b =
  const out = [...(open ? conclusionLive(sig, pos, px) : conclusion(sig, pos)), '
  if (qk.length) out.push('', 'QUICK TRADES, 2 TO 5 DAYS (' + qk.length + ')');
  for (const q of qk) {
    const p = await live(q.sym, q.last), where = p > q.buyHi ? 'ABOVE its entry r
    out.push(q.sym + ' ' + money(p) + ': ' + where + '\n  entry ' + money(q.buyLo
  }
```

```
  if (buys.length) out.push('', 'ENTRY CONFIRMED, swing trades of days to weeks (
  for (const b of buys) {
    const p = await live(b.sym, b.last), where = p > b.buyHi ? 'ABOVE its buy ran
    out.push(b.sym + ' ' + money(p) + ': ' + where + '\n  buy range ' + money(b.b
  }
  const lds = (sig.leaders || []).filter(x => !pos[x.sym] && OK.has(x.sym)).slice
  if (lds.length) { out.push('', 'LEADERS, strongest stocks, trend trades of week
    for (const r of lds) { let p = r.last; if (open) { const j = await liveQ((r.s
      out.push(r.sym + ' ' + money(p) + ': ' + (p > r.buyHi ? 'ABOVE its buy rang
    out.push('Full list and its record: send   leaders'); }
  if (!qk.length && !buys.length && !lds.length) out.push('', 'Nothing passed the
  out.push('', 'These are tested setups, not hot tips. Small wins, strict stops,
  return out.join('\n');
}
// On-demand analysis of ANY ticker, right now, with the same rules as the nightl
let spyDaily = null;
async function loadDaily(sym) {
  if (sym === 'SPY' && spyDaily) return spyDaily;
  const j = await td('/time_series?symbol=' + encodeURIComponent(sym) + '&interva
  const now = Date.now(), all = j.values.map(v => ({ d: v.datetime.slice(0, 10),
  const res = { candles: all.filter(c => barClosed(c.d, now)), last: all[all.leng
}
function addExtra(sym) { const a = readExtra(); if (a.includes(sym)) return false

// DEEP CHECK: everything public about one stock, gathered for  check SYM . Infor
// predict returns in the system's own tests, so none of this can turn a stock in
async function deepInfo(sym) {
  if (!env.FINNHUB_KEY) return [];
  const T = encodeURIComponent(env.FINNHUB_KEY), d = (ms) => new Date(ms).toISOSt
  const out = ['', 'WHAT THE MARKET SAYS (information only; in the tests, news an
  const pf = await get('/stock/profile2?symbol=' + sym); if (pf && pf.name) out.p
  const ec = await get('/calendar/earnings?symbol=' + sym + '&from=' + d(Date.now
  out.push('Next earnings: ' + (nx ? nx.date + (nx.hour === 'bmo' ? ' (before the
  const es = await get('/stock/earnings?symbol=' + sym); if (Array.isArray(es) &&
  const rc = await get('/stock/recommendation?symbol=' + sym); if (Array.isArray(
    out.push('Analysts (' + a.period.slice(0, 7) + '): ' + pos(a) + ' buy, ' + (a
  const it = await get('/stock/insider-transactions?symbol=' + sym + '&from=' + d
  if (tx.length) { const buys = tx.filter(x => x.transactionCode === 'P'), sells
    out.push('Insiders, last 90 days: ' + buys.length + ' open-market buys (' + m
  const nw = await get('/company-news?symbol=' + sym + '&from=' + d(Date.now() -
  if (hs.length) { const nr = negNews(hs); out.push('News this week (' + hs.lengt
  else out.push('News this week: none found.');
  // official company events (SEC) and, for drug makers, late-stage trials (Clini
  { const CK = readJson0('data/sec-ciks.json') || {}, ES = readJson0('data/events
    if (!cik) { const tl = await secJson(SEC_WWW + '/files/company_tickers.json')
    const j = cik ? await secJson(SEC_DATA + '/submissions/CIK' + String(cik).pad
    if (r && r.form) { sic = j.sic || sic; const since = new Date(Date.now() - 90
```

```
      for (let k = 0; k < Math.min(r.form.length, 100) && rows.length < 6; k++) {
        const f = evFams(r.form[k], (r.items || [])[k], sic, ES).filter(q => q !=
        rows.push(pt.slice(0, 10) + ' ' + f.map(q => { const F = ES && ES.fam &&
          return nm + (F && F.warn ? ' [WARNING: such filings were followed by '
      out.push('SEC filings, last 90 days (official): ' + (rows.length ? rows.joi
    else out.push('SEC filings: could not be read (' + (!cik ? 'the SEC company l
    if (/^(283[3-6]|8731)$/.test(String(sic || '')) && j && j.name) { const nm =
      try { const rr = await fetch(CTG + '/api/v2/studies?query.spons=' + encodeU
        const L2 = ((cj && cj.studies) || []).map(x => { const ps = x.protocolSec
        const soon = L2.filter(x => { const t = Date.parse(x.d.length === 7 ? x.d
        if (cj) out.push('Late-stage drug trials (ClinicalTrials.gov, phase 3, sp
  { const hc = halalChecks(sym).text; if (hc) out.push(hc); }
  { const tp = readTips().filter(x => x.sym === sym); if (tp.length) out.push('Yo
  return out;
}
async function checkLive(sym) {
  const d = await loadDaily(sym); if (!d || !d.candles.length) return sym + ': I
  const c = d.candles, i = c.length - 1, out = [sym + ', analysed now from the '
  const refLine2 = () => { const r = L.refPlan(c, d.last), o = L.refOdds(c);
    if (!r) return 'CONCLUSION: no plan. ' + (c.length < 30 ? 'It is too new.' :
    if (o && o.won < 0.25) return 'CONCLUSION: no plan. On its own history the re
    return 'CONCLUSION (reference only, not tested): ' + (r.extended ? 'wait for
  let word = 'NO BUY: no tested rule supports a trade on it.';
  if (c.length < 600) out.push('It has under three years of price history, so no
  else {
    const spy = await loadDaily('SPY'), mkt = spy ? L.marketMap(spy.candles) : nu
    const mom = spy && spy.candles.length > 130 ? spy.candles[spy.candles.length
    const T = L.trendStats(c, ind, mom), Lr = L.learn(c, ind, null, false), halal
    const trig = L.buyTrigger(c, ind, Lr), dist = Lr.v.mult * ind.atr[i], half =
    out.push((T.up ? 'In an uptrend' : c[i].c < ind.e[L.CFG.TREND][i] ? 'In a dow
      'Rule tested for it: ' + L.ruleText(Lr.v) + '. ' + Lr.test.n + ' unseen tra
    if (v.act) word = Lr.test.n < 30 ? 'WATCH ONLY (EARLY EVIDENCE, ' + Lr.test.n
    else if (v.blocked) word = 'NO BUY: ' + v.why;
    else if (Lr.proven && trig.price) word = 'WATCH: not yet. It becomes a buy on
    else if (Q) { const net = netOf(Q.avgPct, d.last); word = okEv(Q.test.n, net,
    else if (Lr.proven) word = 'WATCH: tested rule, but no entry now.';
    if (v.act) out.push('CONCLUSION: ENTRY CONFIRMED. BUY ' + money(d.last) + ' t
    else if (v.blocked) out.push('CONCLUSION: no trade. ' + v.why);
    else if (Lr.proven && trig.price) out.push('CONCLUSION: not yet. BUY only aft
    else if (Q) { const P = L.quickPlan(d.last, ind.atr[i], ind.sma5[i], Q); out.
    else if (Lr.proven) out.push('CONCLUSION: no trade now. ' + trig.text);
    else out.push('No tested rule supports a trade on this stock.', refLine2());
    out.push(L.HALAL_TEXT[halal] + '.');
  }
  try { out.push(...await deepInfo(sym)); } catch (e) {}
  if (myHalal(sym) === 'fail') word = 'NO BUY: you marked it NOT halal in ZAD.';
  out.unshift(sym + ': ' + word);
```

```
  out.push(added ? 'I have added ' + sym + ' to my nightly scan. Send: remove ' +
  return out.join('\n');
}
// ---- Tip log. You forward what a channel or person recommends; the bot stamps
// tested rules, adds it to the nightly scan, and from then on scores that source
const TIP_FILE = 'data/tips.jsonl', NOT_TICKERS = new Set(['USD', 'US', 'USA', 'E
const readTips = () => fs.existsSync(TIP_FILE) ? fs.readFileSync(TIP_FILE, 'utf8'
const western = (t) => t.replace(/[\u0660-\u0669]/g, d => String(d.charCodeAt(0)
function parseTips(raw) {       // source name, tickers, and any "buy below" / "c
  const text = western(raw), first = text.split('\n')[0], m = first.match(/^\s*(?
  let src = m ? m[1].replace(/\b[A-Z]{1,5}\b.*$/, '').replace(/[:\-]+\s*$/, '').t
  // a "Tip NAME source" / "source: NAME" line names the source; its own capital
  const srcLine = text.match(/^\s*(?:tips?\s+)?(.{2,40}?)\s+source\s*$/im) || tex
  const named = srcLine && /[a-z\u0600-\u06FF]/.test(srcLine[1]) ? srcLine : null
  const body0 = m ? text.slice(first.length) + ' ' + m[1] : text, body = named ?
  while ((x = re.exec(body))) if (!NOT_TICKERS.has(x[0]) && !found.some(f => f.sy
  found.forEach((f, n) => { const seg = body.slice(f.at, n + 1 < found.length ? f
    f.below = num(/(?:\u0627\u0642\u0644 \u0645\u0646|\u0623\u0642\u0644 \u0645\u
  // the source's name: one you logged before that appears anywhere in the text w
  { const known = [...new Set(readTips().map(t => t.src))].filter(n => n && n.len
    const line = text.match(/^\s*(?:tips?\s+)?(.{2,40}?)\s+source\s*$/im) || text
    if (named) src = named[1].trim(); else if (known) src = known; else if (line
  return { src: src || 'Unnamed source', tips: found.slice(0, 15) };
}
async function logTips(raw, fwd) {     // fwd = the channel or person a message w
  const parsed = parseTips(raw), tips = parsed.tips, src = parsed.src === 'Unname
  const paid = /\b(sponsored|paid (promotion|partnership|post)|compensated|on beh
  const old = readTips(), day = ny(Date.now()).d, out = ['Logged ' + tips.length
  for (const t of tips) {
    const rep = await checkLive(t.sym), px = rep.match(/Closed ([\d.]+)\./), conc
    const p0 = px ? +px[1] : null;
    if (!p0) { out.push(t.sym + ': not logged. I could not load prices for it, so
    if (!old.some(o => o.sym === t.sym && o.src === src && o.d === day)) fs.appen
    out.push(t.sym + (p0 ? ' at ' + money(p0) : '') + (t.below ? ' (source says b
  }
  out.push('', 'The source gave no stop and no selling price for any of these. I
  return out.join('\n');
}
function tipBoard(sig) {
  const b = sig && sig.tipBoard; if (!b || !b.tips || !b.tips.length) return read
  const pcx = (x) => (x >= 0 ? '+' : '') + (x * 100).toFixed(1) + '%', out = ['TI
  for (const s0 of b.sources) out.push(s0.src + ': ' + s0.n + ' tips, average ' +
  out.push('', 'EACH TIP'); for (const t of b.tips.slice(-20)) out.push(t.sym + '
  return out.join('\n');
}
function checkOne(sym, sig) {
  if (!sig) return 'No scan has run yet.';
```

```
  const q = (sig.quick || []).find(x => x.sym === sym), st = (sig.stocks || []).f
  const word = qOk || stOk ? 'BUY: a tested setup is open. Send  today  for the p
  const lines = [sym + ': ' + (myHalal(sym) === 'fail' ? 'NO BUY: you marked it N
  if (q) lines.push('Quick trade setup: entry ' + money(q.buyLo) + ' to ' + money
  if (st) lines.push('Closed ' + money(st.last) + '. ' + (st.uptrend ? 'In an upt
  if (!q && !st) lines.push('Not on the scanned list, or it has under three years
  return lines.filter(Boolean).join('\n');
}
// Pre-market briefing: sent once each weekday about an hour before the US open,
async function preMarket(sig) {
  if (!sig || !env.TG_TOKEN || !env.TG_CHAT) return;
  const n = ny(Date.now());
  if (!env.BRIEF_NOW && (n.weekend || isHoliday(n.d) || n.mins < 240 || n.mins >
  const st = readJson('data/tg-state.json', { offset: 0 }); if (st.brief === n.d
  if (QUIET()) await telegram(await actionsText(sig, loadPos(), null, 'MORNING AC
  else await telegram(planText(sig, loadPos(), null, 'MORNING PLAN, ' + n.d + '.
  st.brief = n.d; fs.writeFileSync('data/tg-state.json', JSON.stringify(st));
  console.log('pre-market briefing sent');
}
// "market": what is moving right now. Indexes, the eleven sectors ranked by toda
const SECTORS = { XLK: 'Technology', XLF: 'Financials', XLE: 'Energy', XLV: 'Heal
async function marketNow() {
  // Outside forces that move groups of stocks: oil for energy companies, metals
  // Shown as context through exchange-traded funds that follow them. Not a sugge
  const DRIVERS = { USO: 'Crude oil', GLD: 'Gold', SLV: 'Silver', CPER: 'Copper',
  const idx = { SPY: 'S&P 500', QQQ: 'Nasdaq 100', IWM: 'Small companies' }, syms
  for (let k = 0; k < syms.length; k += 7) {            // two batches: the free
    if (k) await sleep(FAST ? 5 : 62000);
    try { const part = syms.slice(k, k + 7), r = await fetch(TD + '/quote?symbol=
  }
  const pcx = (x) => (x >= 0 ? '+' : '') + (x * 100).toFixed(2) + '%', out = ['MA
  out.push(Object.keys(idx).filter(x => q[x] !== undefined).map(x => idx[x] + ' '
  const secs = Object.keys(SECTORS).filter(x => q[x] !== undefined).sort((a, b) =
  if (secs.length) out.push('', 'SECTORS TODAY, strongest first', ...secs.map(x =
  const drv = Object.keys(DRIVERS).filter(x => q[x] !== undefined);
  if (drv.length) out.push('', 'OIL, METALS AND BITCOIN TODAY (context for the st
  if (env.FINNHUB_KEY) {
    const tag = (t) => /\bFDA\b|PDUFA|phase (2|3|ii|iii)\b|trial/i.test(t) ? 'FDA
    for (const cat of ['merger', 'general']) { try { const r = await fetch(FH + '
    heads.sort((a, b) => b.at - a.at);
    const seen = new Set(), top = heads.filter(x => !seen.has(x.h) && seen.add(x.
    out.push('', 'HEADLINES, last 36 hours', ...(top.length ? top.map(x => x.t +
  } else out.push('', 'Headlines are off: the Finnhub key is not passed to this b
  out.push('', 'This is a picture of the moment, not advice on what to buy or sel
  return out.join('\n');
}
// Negative news that blocks a NEW entry for a week. Evidence: disclosed regulato
```

```
// underperformance; drug rejections, going-concern warnings, delisting notices,
// too large to step in front of. Lawsuits and law-firm notices are common and mo
const HARD_NEG = [
  ['Regulator investigation', /\b(SEC|DOJ|FTC|Justice Department|regulators?)\b[^
  ['FDA rejection', /complete response letter|\bCRL\b|FDA (rejects|rejection|decl
  ['Going-concern warning', /going[- ]concern/i],
  ['Delisting notice', /delisting|to be delisted|listing (deficiency|standards)|d
  ['Short-seller report', /short[- ]seller|short report|hindenburg|muddy waters|c
  ['Bankruptcy', /bankruptcy|chapter 11/i],
  ['Accounting restatement', /restat(e|es|ed|ement)\b[^.]{0,40}(financial|results
  // New shares sold to raise money. Evidence: companies that issue stock lag the
  ['Share offering', /\b(pric(es|ed|ing)|announc(es|ed|ing)|launch(es|ed|ing)?|pr
const SOFT_NEG = /class action|lawsuit|\bsue[sd]?\b|securities fraud|shareholder
const LAWFIRM = /shareholder alert|investor alert|investors? (are )?(encouraged|r
const ROUNDUP = /stocks? (mixed|moving|movers)|gap up and gap down|pre-market ses
function negNews(heads) {      // heads: this company's own headlines, newest fir
  const own = heads.filter(h => h && !ROUNDUP.test(h));
  for (const h of own) { if (LAWFIRM.test(h)) continue; const t = HARD_NEG.find((
  const s = own.find(h => SOFT_NEG.test(h));
  return { hard: null, soft: s ? { tag: 'Legal news', h: s.slice(0, 110) } : null
}
async function newsRisk(sym) {
  if (!env.FINNHUB_KEY) return { known: false };
  try { const d = (ms) => new Date(ms).toISOString().slice(0, 10), r = await fetc
    const it = await r.json(); return { known: true, ...negNews((Array.isArray(it
}
// ---- Fast movers requiring investigation. A SEPARATE category from quality can
// investment because it is up 50%; it is a priority to look into. Source: FMP's
async function gainersNow() {
  if (!env.FMP_KEY) return { off: 'no FMP key is passed to the bot' };
  try {
    const r = await fetch(FMPB + '/stable/biggest-gainers?apikey=' + encodeURICom
    if (!r.ok) return { off: 'FMP answered HTTP ' + r.status + ' for the live gai
    const j = await r.json(); if (!Array.isArray(j)) return { off: 'FMP returned
    return { list: j.map(x => ({ sym: String(x.symbol || ''), price: +x.price, pc
  } catch (e) { return { off: 'the live gainers list could not be reached' }; }
}
async function whyMoved(sym, name) {     // the headline must be about THIS compa
  if (!env.FINNHUB_KEY) return '';
  try { const d = (ms) => new Date(ms).toISOString().slice(0, 10), r = await fetc
    const it = await r.json(); if (!Array.isArray(it) || !it.length) return '\n
    const word = String(name || '').split(/[\s,.]+/)[0], esc = (x) => x.replace(/
    const about = new RegExp('\\b' + esc(sym) + '\\b' + (word.length >= 4 ? '|' +
    const tagOf = (h) => { const hn = LAWFIRM.test(h) ? null : HARD_NEG.find(([,
    const tagOf0 = (h) => /offering|private placement|warrants|registered direct/
    const heads = it.map(x => String(x.headline || '')).filter(Boolean), own = he
    if (!pick) return '\n  news: no headline about this company found, only gener
```

```
    return '\n  news: ' + (tagOf(pick) || 'Company news') + ' \u2014 ' + pick.sli
}
const moverEvidence = () => { const m = readJson('data/movers-state.json', null);
const moverLine = async (x) => x.sym + ' ' + (x.pct >= 0 ? '+' : '') + Math.round
async function fastMovers() {     // automatic alert, at most a few a day, each s
  const now = Date.now(); if (!marketOpen(now)) return;
  const st = readJson('data/fm-state.json', { day: '', sent: {}, off: '' }), day
  if (st.off || Object.keys(st.sent).length >= 12) return;
  const g = await gainersNow();
  if (g.off) { st.off = g.off; console.log('fast movers off today: ' + g.off); fs
  const fresh = g.list.filter(x => x.price >= 1 && !st.sent[x.sym]).slice(0, 4);
  const lines = []; for (const x of fresh) { st.sent[x.sym] = 1; lines.push(await
  fs.writeFileSync('data/fm-state.json', JSON.stringify(st));
  if (!QUIET()) await telegram('FAST MOVERS REQUIRING INVESTIGATION\nNot quality
  console.log('fast movers alert: ' + fresh.length);
}
async function moversNow() {      // the "movers" command
  const g = await gainersNow(); if (g.off) return 'Live movers are unavailable: '
  if (!g.list.length) return 'No stock is up 20% or more right now.';
  const main = g.list.filter(x => x.price >= 1), sub = g.list.filter(x => x.price
  return 'FAST MOVERS RIGHT NOW (investigation list, not quality candidates)\n\n'
    + (sub.length ? '\n\nUNDER 1 USD (extreme risk: shown so nothing is hidden, n
}
// ---- Plain questions. No AI: the bot looks for a few words (sell, buy, penny,
const SECTORS_OF = {}; for (const [name, list] of Object.entries({
  'Technology': 'AAPL MSFT NVDA AVGO AMD ORCL CRM ADBE AMAT CSCO IBM INTC INTU NO
  'Consumer discretionary': 'AMZN TSLA HD BKNG LOW MCD NKE SBUX TGT TJX UBER', 'C
  'Financials': 'JPM BAC V MA SPGI', 'Health care': 'UNH LLY JNJ ABBV ABT AMGN BM
  'Energy': 'XOM CVX COP', 'Consumer staples': 'WMT COST PG KO PEP', 'Industrials
const SECTOR_WORDS = [['Technology', /\b(tech|technology|software|semiconductors?
  ['Consumer discretionary', /\b(consumer|retail|discretionary|cars?|autos?)\b/i]
const COMMON_WORDS = new Set('IT THIS THAT MY THE A AN NOW TODAY STOCK STOCKS PEN
function findTicker(text, sig, pos) {
  const known = new Set([...Object.keys(pos || {}), ...((sig && sig.stocks) || []
  const toks = text.replace(/[^A-Za-z.\s-]/g, ' ').split(/\s+/).filter(Boolean),
  for (const t of toks) if (/^[A-Z]{2,5}$/.test(t) && okT(t)) return t;
  for (const t of toks) if (known.has(t.toUpperCase()) && okT(t.toUpperCase())) r
  const m = text.match(/\b(?:sell|buy|check|about|hunt|find)\s+([A-Za-z]{1,5})\b/
}
const universe = (sig) => { const ind = {}; for (const c of ((DISCMETA && DISCMET
  return [...((sig && sig.stocks) || []), ...((sig && sig.uni) || [])].map(x => (
const supportLine = (x, sig) => { const q = ((sig && sig.quick) || []).find(k =>
  return x.state === 'Buy watch' && x.buyLo ? x.sym + ': ENTRY CONFIRMED. BUY ' +
    : q ? x.sym + ': QUICK TRADE. BUY ' + money(q.buyLo) + ' to ' + money(q.buyHi
    : x.proven && x.trigger && x.trigger / x.last < 1.15 ? x.sym + ': BUY WATCH.
function sectorReply(name, sig) {
  const rows = universe(sig).filter(x => x.sector.toLowerCase().includes(name.toL
```

```
  if (!rows.length) return name + ': I have no stocks from this sector on my scan
  const sup = rows.map(x => supportLine(x, sig)).filter(Boolean), up = rows.filte
  return [name.toUpperCase() + ', from the ' + sig.session + ' close: ' + rows.le
}
// OPINION CHECK: a pasted or forwarded opinion with no tickers (a market call, a
// so it is answered with what can be checked: its warning signs, what the system
// and which stocks in those themes have tested support today.
const RED_FLAGS = [
  [/(المال|افلوسك|أموالك|اموالك|كل (استثمر بالكامل|all[\s-]?in\b|invest (everythin
  [/ولا تقلق|لا تقلق|don'?t worry|no risk|بدون مخاطر|مضمون|guarantee/i, 'It tells y
  [/(إلى|الى|حتى) عام\s*\S{2,4}|until\s+20\d\d|لا يمكن ايقاف|لا يمكن إيقاف|unstop
  [/متلاعب|manipulat/i, 'It blames every drop on manipulators in advance, so you w
  [/رائع [فرص]ة|الوقت)|حان (وقت|now is the time|don'?t miss|لا تفوت/i, 'It pushes
  [/استمتع|سعداء|مذهل|amazing|incredible|!!/i, 'It runs on excitement, not eviden
const THEMES = [
  ['AI, chips, memory and data centres', /اصطناعي|ذكاء|\bai\b|مراكز البيانات|data
  ['Energy and power', /الطاقه|طاقة|energy|power|نفط|oil|كهرباء|electric/i, 'Ener
  ['Shipping and transport', /شحن|shipping|ناقلات|tankers?|transport/i, 'Industria
const THEME_PROXY = { 'AI, chips, memory and data centres': 'SMH', 'Energy and po
const OPIN_FILE = 'data/opinions.jsonl';
const readOpinions = () => fs.existsSync(OPIN_FILE) ? fs.readFileSync(OPIN_FILE,
// Does the message touch what the system is working on right now: your positions
const themeOfSector = (sec) => !sec ? null : /semi|software|tech|electron|communi
async function opinionReply(text, sig, src) {
  const flags = RED_FLAGS.filter(([re]) => re.test(text)).map(([, w]) => w), them
  const now = Date.now(), day = ny(now).d, past = readOpinions(), tnames = themes
  // 1. REPEATED: has this theme been pushed before, by whom, and what has it don
  const proxyNow = {}; for (const t of tnames) { const p = THEME_PROXY[t]; if (p)
  fs.appendFileSync(OPIN_FILE, JSON.stringify({ d: day, ts: now, src: src || 'unn
  const repeat = tnames.map(t => { const prev = past.filter(o => (o.themes || [])
    return prev.length ? t + ': pushed ' + (prev.length + 1) + ' times in 30 days
  // 2. RELEVANT TO WHAT I AM WORKING ON: your positions, tonight's buys, today's
  const SC = readJson('data/sectors.json', {}), pos = loadPos(), touch = [];
  if (env.FINNHUB_KEY) { let got = 0; for (const sym of Object.keys(pos)) if (SC[
  for (const sym of Object.keys(pos)) { const th = themeOfSector(SC[sym]); if (th
  for (const c of sig ? planCands(sig, pos).filter(x => x.ok) : []) { const th =
  { const rd = readJson('data/intraday-radar.json', null); if (rd && rd.session =
  // 3. CLAIMS THAT CAN BE CHECKED: "earnings season" -> your stocks reporting in
  const earn = [];
  if (/أرباح|ارباح|نتائج|إعلانات|اعلانات|earnings|results|guidance/i.test(text) &&
    try { const r = await fetch(FH + '/calendar/earnings?from=' + day + '&to=' +
      if (e && e.date) earn.push(sym + ' (yours) reports on ' + e.date + '. A res
  const out = ['OPINION CHECK' + (src ? ' (' + src + ')' : '') + ': no stock is n
  if (flags.length) out.push('', 'WARNING SIGNS (' + flags.length + ')', ...flags
  const mk = sig && sig.market ? sig.market : null, fair = readJson('data/leaders
  out.push('', 'WHAT MY DATA SAYS');
  if (mk) out.push('- S&P 500: ' + (mk.up ? 'in an uptrend' : 'in a downtrend') +
```

```
  if (fair && fair.liveVerdict) out.push('- Buying the strongest stocks (LEADERS)
  if (ws && ws.base && ws.base.newer && ws.base.newer.n) out.push('- A random sto
  for (const [name, , sec] of themes) {
    const rows = sig ? universe(sig).filter(x => x.sector && (x.sector.toLowerCas
    const sup = rows.map(x => supportLine(x, sig)).filter(Boolean), up = rows.fil
    out.push('', name.toUpperCase() + ': ' + (rows.length ? rows.length + ' on my
  }
  if (touch.length) out.push('', 'WHAT THIS TOUCHES IN MY WORK RIGHT NOW', ...tou
  if (repeat.length) out.push('', 'REPEATED', ...repeat);
  if (earn.length) out.push('', 'CHECKED CLAIM: earnings season', ...earn);
  out.push('', 'WHAT TO DO: nothing changes. Keep your stops, buy only from the p
  return out.join('\n');
}
async function pennyReply(sig) {
  const rows = universe(sig).filter(x => x.last < 5), sup = rows.map(x => support
  const g = marketOpen(Date.now()) ? await gainersNow() : { list: [] };
  const pm = (g.list || []).filter(x => x.price < 5).slice(0, 5);
  if (pm.length) out.push('', 'MOVING NOW under 5 USD (investigation only, no pri
  const m = readJson('data/movers-state.json', null);
  out.push('', m && m.penny ? 'What my own radar measured over the last ' + m.ses
  return out.join('\n');
}
async function sellReply(sym, sig, pos) {
  const P = pos[sym];
  if (P) { let p = null; if (marketOpen(Date.now())) { const j = await liveQ((sym
    const pv = (sig && sig.posVerdict || {})[sym], lines = ['WHEN TO SELL ' + sym
    if (p && P.stop && p <= P.stop) lines.push('SELL NOW: your stop ' + money(P.s
    else if (pv) lines.push('SELL at the next open: ' + pv);
    else lines.push('Not yet. HOLD.');
    // THE SELL PLAN, in the order that matters: the tested exit first (it is wha
    let c = null; try { const d = await loadDaily(sym); c = d && d.candles; } cat
    const avgN = (n) => c && c.length >= n ? c.slice(-n).reduce((a, x) => a + x.c
    if (P.kind === 'quick') { const a5 = avgN(5); lines.push('SELL (TESTED EXIT):
    else if (P.kind === 'entry') { const lv = P.exitMA ? avgN(P.exitMA) : P.exitB
    else if (P.kind === 'leader') lines.push('SELL (TESTED EXIT): on a daily clos
    else lines.push('Your own pick: no tested exit. I check it every night with t
    lines.push('STOP: ' + (P.stop ? money(P.stop) + ' (sell at once if it trades
    lines.push('TARGET: ' + (P.t1 ? money(P.t1) + (P.ownT ? ' (yours)' : P.tmode
    return lines.join('\n'); }
  const d = await loadDaily(sym); if (!d || d.candles.length < 60) return sym + '
  const c = d.candles, i = c.length - 1, ind = L.indicators(c, null), a = ind.atr
  return ['WHEN TO SELL ' + sym + ' (closed ' + money(c[i].c) + '). I am not trac
}
// HOLD OR SELL REVIEW of a stock you own (bought before the system or not). It s
// day when it was in the same state as now (trend, and how stretched it was abov
// next 20 sessions, against the S&P 500 on the same days. Then it applies the ho
// ends the trend (sell); a very stretched stock whose own past says it lagged fr
```

```
async function reviewReply(sym, sig, pos) {
  const d = await loadDaily(sym); if (!d || d.candles.length < 260) return sym +
  const spy = await loadDaily('SPY'), sMap = new Map((spy ? spy.candles : []).map
  const c = d.candles, n = c.length, i = n - 1, cl = c.map(x => x.c);
  const sma = (p) => { const o = new Array(n).fill(null); let t = 0; for (let k =
  const m50 = sma(50), m200 = sma(200), tr = c.map((x, k) => k ? Math.max(x.h - x
  const a14 = (() => { const o = new Array(n).fill(null); let t = 0; for (let k =
  const bucket = (k) => { const z = a14[k] ? (cl[k] - m50[k]) / a14[k] : 0; retur
  const H = 20, now = bucket(i), same = [];
  for (let k = 200; k < n - H; k += 5) if (bucket(k) === now) { const r = cl[k +
  const avgOf = (a) => a.length ? a.reduce((x, y) => x + y, 0) / a.length : null,
  let hi52 = 0; for (let k = Math.max(0, n - 252); k < n; k++) hi52 = Math.max(hi
  const z = a14[i] ? (cl[i] - m50[i]) / a14[i] : 0, P = pos[sym];
  let p = null; if (marketOpen(Date.now())) { const j = await liveQ(sym); p = +j.
  const px = p || cl[i], trail = +Math.max(cl[i] - 2 * a14[i], m50[i] < cl[i] ? m
  let earn = ''; if (env.FINNHUB_KEY) { try { const day = new Date().toISOString(
  const SS = readJson('data/score-study.json', null), sc = SS && SS.scores && SS.
  const enough = same.length >= 20, lag = enough && hs !== null && hr < hs, neg =
  let verdict, why;
  if (cl[i] < m50[i] && cl[i] < m200[i]) { verdict = 'SELL'; why = 'the trend is
  else if (cl[i] < m50[i]) { verdict = neg || lag ? 'SELL' : 'SELL OR TIGHTEN'; w
  else if (z >= 4 && (neg || lag)) { verdict = 'TAKE PART PROFIT'; why = 'it is v
  else if (z >= 4) { verdict = 'HOLD, RAISE THE STOP'; why = 'strong but very str
  else { verdict = 'HOLD'; why = 'the trend is intact: above its 50 day' + (cl[i]
  // a SYSTEM trade (quick, swing, radar, leader) follows its own tested exit and
  const sysTrade = P && ['quick', 'entry', 'radar', 'leader'].includes(P.kind), p
  if (sysTrade && pv) { verdict = 'SELL (its tested exit fired)'; why = pv.replac
  else if (sysTrade && !verdict.startsWith('SELL')) { verdict = 'HOLD (follow its
  const out = ['HOLD OR SELL: ' + sym + ' → ' + verdict, 'Price ' + money(px) + (
    'Why: ' + why,
    'Its own past: ' + (enough ? same.length + ' earlier times in this same state
    'From its 1-year high: ' + pcs(px / hi52 - 1) + (sc !== undefined && sc !== n
    sysTrade ? 'Stop to use: ' + money(P.stop) + ' (the trade\u2019s own tested s
  if (earn) out.push('Earnings on ' + earn + ': a result can jump the price past
  if (!P) out.push('', 'I am not tracking it. Send:  bought ' + sym + ' YOUR-PRIC
  else if (!sysTrade && (verdict === 'HOLD' || verdict.startsWith('HOLD,'))) out.
  out.push('Tested on its own history, not a promise. Not financial advice, not a
  return out.join('\n');
}
// Is this stock on today's tested BUY list? (a tested rule fired on it at the la
const buyToday = (sym, sig) => { if (!sig) return null;
  const q = (sig.quickPaused ? [] : sig.quick || []).find(x => x.sym === sym && !
  const r = q || b; return r ? { lo: r.buyLo, hi: r.buyHi, stop: r.stop, n: r.n,
// YOUR STOCKS: one clear action per stock you own. SELL ALL, SELL HALF, SELL SOM
// Built from the HOLD OR SELL study of each stock. "Buy a little more" only when
// never just to lower the average of a falling stock.
async function holdingsText(sig, pos, title, raw) {
```

```
  const mine = Object.keys(pos); if (!mine.length) return raw ? { acts: { sell: [
  const pv = privRead(), out = [title || 'YOUR STOCKS: what to do'], acts = { sel
  const mark = (sym, act, why) => { if (pos[sym]) pos[sym].lastAct = { act, why:
  for (const sym of mine.slice(0, 15)) { let r = ''; try { r = await reviewReply(
    const P = pos[sym], verdict = ((r.match(/→ (.+)/) || [])[1] || '').trim(), wh
    const pm = r.match(/^Price ([\d.,]+)/m), px = pm ? +pm[1].replace(/,/g, '') :
    const ew = eventWarn(sym), tail = (earn ? ' | EARNINGS ' + earn + ': decide b
    // A trade the system chose follows ITS OWN tested exit and stop. The general
    // is for your own picks only: a dip-buy is below its short averages by desig
    if (['quick', 'entry', 'radar', 'leader'].includes(P.kind)) {
      const stopHit = P.stop && px && px <= P.stop, xf = P.exitFlag && (!P.since
      if (stopHit || xf) { mark(sym, 'SELL ALL', stopHit ? 'stop reached' : 'its
      const ex = P.kind === 'quick' ? 'exit: first close ' + (P.q === 'gap' ? 'be
      mark(sym, 'HOLD', 'its own rule'); acts.hold.push(sym + chg + ': HOLD, stop
      (acts.holdShort = acts.holdShort || []).push(sym + chg + ' stop ' + (P.stop
    if (P.stop && px && px <= P.stop) { mark(sym, 'SELL ALL', 'stop reached'); ac
    else if (/^SELL( \(|$)/.test(verdict)) mark(sym, 'SELL ALL', why), acts.sell.
    else if (verdict === 'SELL OR TIGHTEN') mark(sym, 'SELL HALF', why), acts.par
    else if (verdict === 'TAKE PART PROFIT') mark(sym, 'SELL SOME', 'lock profit'
    else { mark(sym, 'HOLD', why); const b0 = buyToday(sym, sig), b = b0 && !b0.t
      if (b && verdict.startsWith('HOLD') && !(P.stop && px && px <= P.stop)) act
      else { acts.hold.push(sym + chg + ': HOLD, stop ' + stop + (verdict.include
  if (raw) return { acts, holdShort: acts.holdShort || [] };
  if (acts.sell.length) out.push('', 'SELL ALL', ...acts.sell);
  if (acts.part.length) out.push('', 'SELL PART', ...acts.part);
  if (acts.add.length) out.push('', 'BUY A LITTLE MORE', ...acts.add);
  if (acts.hold.length) out.push('', 'HOLD (keep these stops at your broker)', ..
  { const dn = buyDayNote(), tn = timingNote(); if (tn) out.push('', tn); if (dn)
  { const g = goalsText().split('\n'); if (/^MONEY GOALS \((?!they start)/.test(g
  out.push('', 'After you act, send:  sold SYM PRICE   or  sold SYM PRICE 5 share
  return out.join('\n');
}
// ACTIONS: the ONE message to act on. Sells first, then buys (strongest first, w
// holds and the goal. Everything else the system sends is information. Send  det
async function actionsText(sig, pos, px, title) {
  const h = await holdingsText(sig, pos, null, true), plan = planText(sig, pos, p
  const out = [title || 'ACTIONS'];
  const planSells = (plan.match(/\nSELL\n([\s\S]*?)(\n\n|$)/) || [])[1] || '', so
  const sells = [...(planSells ? planSells.split('\n') : []), ...h.acts.sell.filt
  if (sells.length) out.push('', 'SELL', ...sells.map(l => '- ' + l));
  if (h.acts.part.length) out.push('', 'SELL PART', ...h.acts.part.map(l => '- '
  const buyPart = plan.replace(/\nSELL\n[\s\S]*?(\n\n|$)/, '\n').trim();
  out.push('', buyPart.replace(/^\n+/, ''));
  if (h.acts.add.length) out.push('', 'ADD TO A STOCK YOU OWN', ...h.acts.add.map
  { const z = readJson0('data/zad-list.json'), bad = z ? Object.keys(pos || {}).f
  if (h.holdShort.length) out.push('', 'HOLD (stops at your broker): ' + h.holdSh
  { const dn = buyDayNote(), tn = timingNote(); if (tn) out.push(tn); if (dn) out
```

```
  { const g = goalsText().split('\n'); if (/^MONEY GOALS \((?!they start)/.test(g
  { const cc = planCands(sig, pos || {}), w = cc.filter(x => x.watch), rb = cc.fi
    { const ct = calendarText(true); if (ct) out.push(ct); }
    { const ml = macroLines(pos, cc); if (ml.length) out.push(...ml); const mb =
    { let ml = null; try { const L = fs.readFileSync('data/macro-log.jsonl', 'utf
    if (nb.length) out.push('HELD BACK (BAD NEWS): ' + nb.slice(0, 8).map(x => x.
  if (sig && sig.quickHeld && sig.quickHeld.length) out.push('QUICK TRADES held b
  { const HL = readJson('data/hunting-lab.json', null), LS = readJson('data/leade
    const SM = readJson('data/smart-money-study.json', null), smUp = SM && SM.ris
    const rOnly = rows.filter(r => r.market && r.market.risingOnly).map(r => r.na
    if (rows.length || LS) out.push('RESEARCH (Sunday): ' + (rows.length ? (pass.
  out.push('', 'This is the one message to act on. Check ZAD, buy inside the rang
  return out.join('\n');
}
// TRADE WORDS: turns the ways you might say a buy or a sale into the standard li
// "I bought 10 shares of RELL at 21.19", "bought 10 RELL @ 21.19", "اشتريت RELL
// "sold 5 RELL at 23" ... become "bought RELL 21.19 10 shares" / "sold TTD 12.10
// common typos for "shares" (shears, shars, shrs, sahres...) are read as shares
const SHARE_TYPOS = /\b(?:shears?|shars?|shres?|shrs?|sahres?|shaers?|shaes?|shar
function tradeWords(t0) {
  let t = western(t0).replace(/[،,]/g, ' ').replace(/\s+/g, ' ').trim();
  if (/^(?:i\s+)?(?:bought (?:more|extra)|added|add more)\b/i.test(t)) return t0;
  const buyW = /^(?:i\s+)?(?:just\s+)?(?:(?:bought|purchased)\b|اشتريت|شريت|)/i, s
  const isBuy = buyW.test(t), isSell = sellW.test(t); if ((!isBuy && !isSell) ||
  if (/^(?:i\s+)?(?:bought|sold)\s+[A-Za-z.\-]{1,6}(\s|$)/i.test(t) && !/\b(of|@|
  const half = /\bhalf\b|نص|نصف/i.test(t), all = /\ball\b|كل/i.test(t);
  const words = t.replace(buyW, '').replace(sellW, '').replace(/\b(all|my|the|of|
  const sym = (words.find(w => /^[A-Za-z]{1,5}(\.[A-Za-z])?$/.test(w) && !NOT_TIC
  const nums = (t.match(/\d+(?:\.\d+)?/g) || []).map(Number);
  // the price is the number written after at/@/بسعر, or the number with decimals
  let price = null, sh = null;
  { const pm = t.match(/(?:@|\bat\b|بسعر|على)\s*(\d+(?:\.\d+)?)/i); if (pm) price
  { const sm = t.match(/(\d+)\s*(?:shares?|sh|stocks?|أسهم|اسهم|سهم)/i) || t.matc
  if (price === null) { const dec = nums.find(n => !Number.isInteger(n)); price =
  if (sh === null && nums.length >= 2) sh = nums.find(n => n !== price && Number.
  const rest = (t.match(/\b(stop\s+\d+(?:\.\d+)?|target\s+\d+(?:\.\d+)?)/gi) || [
  if (isBuy) return 'bought ' + sym + (price ? ' ' + price : '') + (sh ? ' ' + sh
  if (half && !all) return 'sold half ' + sym + (price ? ' ' + price : '');
  return 'sold ' + sym + (price ? ' ' + price : '') + (sh && !all ? ' ' + sh + '
}
// ONE-TAP RECORDING: buttons under ACTIONS and BUY NOW so the system records you
// Tapping "Bought" records the buy at the live price with the recommended number
// "Skipped" records that you passed on it (missed trades are evidence too). If y
function tradeButtons(sig, pos, max) {
  const rows = [], c = sig ? planCands(sig, pos).filter(x => x.ok) : [];
  for (const x of c.slice(0, max || 5)) { const a = amountFor(x, sig.market, fals
  for (const sym of Object.keys(pos || {}).slice(0, 6)) rows.push([{ text: 'Sold
```

```
  return rows;
}
const QUIET = () => { const st = readJson0('data/settings.json') || {}; return st
// "Should I buy more of X?" for a stock you already hold. The answer follows the
async function addReply(sym, pos) {
  const P = pos[sym]; let p = null;
  if (marketOpen(Date.now())) { const j = await liveQ((sym)); p = +j.price || nul
  const now = p ? ' You bought at ' + money(P.entry) + ', now ' + money(p) + ' ('
  const plan = 'Your plan is unchanged: HOLD | STOP ' + (P.stop ? money(P.stop) :
  const SG = readJson('data/signals.json', null), out = ['SHOULD YOU BUY MORE ' +
  if (b && !b.thin && p && p > P.entry * 1.01 && !(P.since && b0s && b0s <= P.sin
  else if (b && b.thin) out.push('No.' + now, 'A buy rule fired, but it is only E
  else if (p && P.stop && p <= P.stop) out.push('No. It has reached your stop at
  else if (p && p < P.entry) out.push('No.' + now, 'It is below your entry. Buyin
  else if (P.kind === 'quick') out.push('No.' + now, 'A quick trade is a single e
  else if (p) out.push('Only as a planned second stage.' + now, 'It is above your
  else out.push('The market is closed, so I cannot see the price.' + now, 'The ru
  return out.join('\n');
}
// "From SOURCE what to buy?"  Lists what that source recommended, and which of i
function sourceReply(src, sig) {
  const mine = readTips().filter(t => t.src === src), seen = new Set(), tips = mi
  const rows = universe(sig), board = (sig && sig.tipBoard) || { tips: [], source
  const good = [], rest = [];
  for (const t of tips) {
    const x = rows.find(r => r.sym === t.sym), sup = x ? supportLine(x, sig) : nu
    const since = perf && perf.ret !== null ? ' Since the tip: ' + pcx(perf.ret)
    if (sup) good.push(sup + (t.below ? ' | the source said: buy below ' + money(
    else rest.push(t.sym + ': ' + (x ? 'no tested support today' : 'not scanned y
  }
  const rec = board.sources.find(g => g.src === src);
  return ['FROM ' + src + ': ' + tips.length + ' stock' + (tips.length === 1 ? ''
    rec ? 'This source\u2019s record so far: ' + rec.n + ' tip' + (rec.n === 1 ?
    'A source naming a stock is a lead. Only a line starting ENTRY CONFIRMED, QUI
}
async function natural(text, sig, pos) {
  if (/^(i\s+)?(bought|sold)\b|^(check|add|analy[sz]e|remove|delete|unwatch|stop|
  { const low = text.toLowerCase(), srcs = [...new Set(readTips().map(t => t.src)
  if (/\bleaders?\b|\bstrongest\b|\bwaves?\b|\btrending\b|ride\b/i.test(text) &&
  if (/\bpenn(y|ies)\b|under\s*\$?\s*5\b|\bcheap stocks?\b/i.test(text)) return p
  const sec = SECTOR_WORDS.find(([, re]) => re.test(text)), tk = findTicker(text,
  if (sec && !tk && sig) return sectorReply(sec[0], sig);
  if (/\bmy (stocks|shares|portfolio|positions)\b|\bholdings\b|\bportfolio\b|what
  if (/^review\b|\bhold or sell\b|\bkeep or sell\b|\breach(ed)? (its|their|the) (
    if (tk) return reviewReply(tk, sig, pos);
    const mine = Object.keys(pos); if (!mine.length) return 'You have no position
    const parts = []; for (const x of mine.slice(0, 8)) parts.push(await reviewRe
```

```
    return parts.join('\n\n') + (mine.length > 8 ? '\n\nOnly the first 8 shown. S
  }
  if (tk && pos[tk] && /\b(more|add|adding|average|averaging|double)\b/i.test(tex
  if (/\bsell\b|\bexit\b|take profit|get out/i.test(text)) return tk ? sellReply(
  if (!tk && /\bbuy\b|\bhunt\b|\brecommend/i.test(text)) return hotList(sig);
  if (tk && /\bbuy\b|\bhunt\b|\bfind\b|look (for|at|into)|what about|how about|\b
  return null;
}
// ---- Watchdog. The bot wakes every few minutes, so it can notice when one of t
// (crashed, was cancelled, or never started) and tell you once, instead of you d
async function watchdog(sig) {
  if (!env.TG_TOKEN || !env.TG_CHAT) return;
  const now = Date.now(), n = ny(now);
  let expected = ''; for (let k = 0; k < 10; k++) { const x = ny(now - k * 864000
  if (!expected) return;
  const later = n.d > expected, st = readJson('data/tg-state.json', { offset: 0 }
  const late = (key, have, due, what, name) => { if (due && (have || '') < expect
  late('scan', sig && sig.session, later || n.mins >= 1230, 'The nightly scan', '
  late('disc', DISCMETA && DISCMETA.session, later && n.mins >= 150, 'The discove
  late('movers', (readJson('data/movers-state.json', {}) || {}).session, later &&
  if (!out.length) return;
  st.warned = warned; fs.writeFileSync('data/tg-state.json', JSON.stringify(st));
  await telegram('WATCHDOG\n' + out.join('\n\n') + '');
  console.log('watchdog: ' + out.length + ' warning(s) sent');
}
async function tgCommands(sig) {
  if (!env.TG_TOKEN || !env.TG_CHAT) return;
  const state = readJson('data/tg-state.json', { offset: 0 }), pos = loadPos(), o
  let ups = [];
  try { const r = await fetch(TG + '/bot' + env.TG_TOKEN + '/getUpdates?timeout='
  // One message may carry several commands, one per line (for example four "foll
  // A leading dash, bullet or backtick copied from a list is ignored. A multi-li
  const CMD_LINE = /^(follow|unfollow|bought|i bought|sold|i sold|stop|target|che
  for (const u of ups) {
    if (u.callback_query) {   // a tapped button: Bought / Sold / Skipped
      const cq = u.callback_query, chatId = cq.message && cq.message.chat ? cq.me
      const [kind, sym, sh] = String(cq.data || '').split('|');
      if (String(chatId) === String(env.TG_CHAT) && sym) {
        if (kind === 'b') { const j = await liveQ(sym), p = +j.price || null; if
        else if (kind === 'x') txt = 'sold ' + sym;
        else if (kind === 's') { let px = null; try { const j = await liveQ(sym);
          fs.appendFileSync('data/skipped.jsonl', JSON.stringify({ sym, d: ny(Dat
          // ask why, with one tap: the system later shows whether your skips wer
          try { await fetch(TG + '/bot' + env.TG_TOKEN + '/sendMessage', { method
        else if (kind === 'r') { fs.appendFileSync('data/skipped.jsonl', JSON.str
      }
      try { await fetch(TG + '/bot' + env.TG_TOKEN + '/answerCallbackQuery', { me
```

```
    if (txt) queue.push({ update_id: u.update_id, message: { chat: { id: chatId
    continue;
  }
  const m = u.message, lines = m && m.text ? m.text.split('\n').map(clean).filt
  if (lines.length > 1 && lines.every(x => CMD_LINE.test(x))) lines.forEach(x =
  else { if (m && m.text && lines.length === 1) m.text = lines[0]; queue.push(u
}
if (state.menuV !== MENU_V) { await sendMenu(); state.menuV = MENU_V; }   // th
for (const u of queue) {
  state.offset = u.update_id + 1;
  const m = u.message; if (!m || !m.text || String(m.chat.id) !== String(env.TG
  { const sl = m.text.trim().match(/^\/([a-z_]+)(?:@\w+)?\s*([\s\S]*)$/i); if (
    m.text = sl[2] ? (/^(check|bought|sold)$/i.test(sl[1]) ? sl[1].toLowerCase(
  { const t0 = m.text.trim().toLowerCase(); if (ASK[t0]) { out.push(ASK[t0]); c
    if (/^(news ?watch|my news|news alerts?|threats?)\s*\??$/.test(t0)) { out.p
    if (/^(market today|today'?s market|day|day type|what (kind of )?day( is (i
  const text = tradeWords(m.text.trim().replace(/\$/g, '').replace(SHARE_TYPOS,
  let k;
  { const tk = (text.match(/\b[A-Z]{2,5}\b/g) || []).filter(x => !NOT_TICKERS.h
    if (!tk.length && !/^\s*(?:tips?|log)\b/i.test(text) && (text.length >= 160
      const src = (m.forward_origin && (m.forward_origin.sender_user_name || (m
      out.push(await opinionReply(text, sig, src)); continue; } }
  { const h1 = text.match(/^(?:halal|zad)\s+([a-z.\-]{1,6})\s+(yes|ok|halal|pas
    const hm = h1 ? [h1[1], !/^(yes|ok|halal|pass)$/i.test(h1[2])] : h2 ? [h2[1
    if (hm) { const sym = hm[0].toUpperCase(), mk = HALALM(); mk[sym] = { s: hm
      out.push(sym + (hm[1] ? ': marked NOT HALAL (ZAD). I will not offer it as
  if (!(m.forward_origin || m.forward_from_chat)) { const nl = await natural(te
  if (/^(sources?|tips?|scoreboard)\s*$/i.test(text)) { out.push(tipBoard(sig))
  if ((k = m.text.trim().match(/^follow\s+(.+)$/i))) {       // follow NAME UCx
    let id = (k[1].match(/UC[\w-]{20,24}/) || [])[0];
    const handle = (k[1].match(/@([\w.-]{3,40})/) || [])[1], name = k[1].replac
    if (!id && handle && env.YT_KEY) {      // a channel link with an @name: as
      try { const r = await fetch(YTAPI + '/youtube/v3/channels?part=snippet&fo
      if (!id) { out.push('I could not find a YouTube channel for @' + handle +
    }
    if (!id || !name) out.push('Send it like this:\nfollow ZipTrader UCxxxxxxxx
    else {    // ask YouTube whose channel this is. If YouTube does not answer,
      const f = await ytFeed(id), a = readChannels().filter(c => c.id !== id &&
      if (f.ok) { const real = String(f.channel || '').slice(0, 60), latest = S
        a.push({ name, id }); fs.writeFileSync(CH_FILE, JSON.stringify(a.slice(
        out.push('Following ' + name + '.\nYouTube says this ID belongs to: ' +
      else if (env.YT_KEY && f.status === 404) out.push('Not followed: YouTube\
      else { a.push({ name, id, unverified: true }); fs.writeFileSync(CH_FILE,
        out.push('Saved ' + name + ', but NOT verified: YouTube (' + f.via + ')
    }
    continue;
  }
```

```
if ((k = m.text.trim().match(/^unfollow\s+(.+)$/i))) { const a = readChannels
if (/^channels?\s*$/i.test(text)) { const a = readChannels(); out.push(a.leng
if ((k = m.text.trim().match(/^(?:name|rename|source)\s+(.{1,80})$/i))) {
  const parts = k[1].split(/\s*=\s*/), from = parts.length > 1 ? parts[0].tri
  if (!to || !hit) out.push(hit ? 'Send it like this: name SOURCE NAME' : 'I
  else { fs.writeFileSync(TIP_FILE, all.map(t => JSON.stringify(t.src === fro
  continue;
}
if (/^\s*(?:tips?\b|log\b|\u062A\u0648\u0635\u064A\u0627\u062A|\u062A\u0648\u
if (/^(leaders?|strongest)\b/i.test(text)) { out.push(leadersText()); continu
if (/^(details?|more|full)\s*$/i.test(text)) { const ps = loadPos(), px = {};
if (/^(record|my record|results?|performance)\s*$/i.test(text)) { out.push(my
if (/^(lessons?|learn\w*|mistakes?)\s*$/i.test(text)) { out.push(lessonsText(
if ((k = text.match(/^(account|capital|balance)\s+(\d+(?:\.\d+)?)\s*(usd|\$)?
if ((k = text.match(/^(?:fee|fees|commission)\s+(\d+(?:\.\d+)?)\s*(usd|\$)?\s
if ((k = text.match(/^risk\s+(\d+(?:\.\d+)?)\s*(usd|\$)?\s*$/i))) { const st
if ((k = text.match(/^score(?:\s+([a-z.\-]{1,6}))?\s*$/i))) { const w = readJ
  if (!w || !w.text) out.push('The stock score runs every Sunday with the fai
  else if (!k[1]) out.push(w.text);
  else { const sy = k[1].toUpperCase(), sc = (w.scores || {})[sy], all = Obje
    out.push(sc === undefined ? sy + ': no score (under 5 USD, thinly traded,
  continue; }
if (/^(lab|hunting|hunting lab|hunt)\s*$/i.test(text)) { const w = readJson('
if (/^(winners?|winner study|patterns?)\s*$/i.test(text)) { const w = readJso
if (/^(report|scan|full report)\s*$/i.test(text)) { const r = fs.existsSync('
if (/^(radar|early|wave|ladder)\b/i.test(text)) { const r = readJson('data/in
if (/^(today|hot|hot\s*list|quick|setups?|ideas?|list today|update|recommend\
if (/^(movers?|gainers?|fast|pennies|penny)\b/i.test(text)) { out.push(await
if (/^(market|news|now|sectors?|breaking)\b/i.test(text) || /what.*(happening
if ((k = text.match(/^(?:check|ask|about|add|analy[sz]e)\s+([a-z.\-]{1,6})\b/
if ((k = text.match(/^(?:remove|delete|unwatch)\s+([a-z.\-]{1,6})\b/i))) { co
// ADDED: more shares of a stock you already hold. The average price and shar
if ((k = text.match(/^(?:i\s+)?(?:added|add more|bought more|bought extra)\s+
  const sym = k[1].toUpperCase(), price = +k[2], P = pos[sym], shm = text.mat
  if (!P) { out.push('I am not tracking ' + sym + ' yet. For a first buy send
  if (!add) { out.push('How many shares did you add? Send:  added ' + sym + '
  const pv = privRead(), old = pv.shares && pv.shares[sym];
  if (!old) { out.push('I do not know how many ' + sym + ' shares you had bef
  const tot = old + add, avg = +((old * P.entry + add * price) / tot).toFixed
  pv.shares[sym] = tot; pv.buys = (pv.buys || []).concat({ sym, usd: +(add *
  out.push('Added ' + add + ' ' + sym + ' at ' + money(price) + '. You now ho
  P.entry = avg; P.addedOn = ny(Date.now()).d; continue;
}
// SOLD PART: "sold SYM PRICE 5 shares" when you sell only some of your share
{ const pm = text.match(/^(?:i\s+)?sold\s+(?!half\b)([a-z.\-]{1,6})\s+(?:at\s
  if (pm && pos[pm[1].toUpperCase()]) { const sym = pm[1].toUpperCase(), exit
    if (held && n0 < held) {
```

```
      fs.appendFileSync(MY_TRADES, JSON.stringify({ sym, kind: (P.kind || 'ma
      const usd = n0 * (exit - P.entry) - 2 * FEE(); pv.shares[sym] = held -
      out.push('Sold ' + n0 + ' of your ' + held + ' ' + sym + ' at ' + money
      continue; } } }
if ((k = text.match(/^(?:i\s+)?bought\s+([a-z.\-]{1,6})(?:\s+(?:at\s+)?(\d+(?
  const sym = k[1].toUpperCase(); let entry = k[2] ? +k[2] : null;
  const shm = text.match(/(\d+(?:\.\d+)?)\s*(?:shares?|sh|stocks?|سهم|اسهم)\b
  if (!entry) { const j = await liveQ((sym)); entry = +j.price || null; }
  if (!entry) { out.push(sym + ': I could not get a price. Send it with the p
  const P = { entry, since: ny(Date.now()).d, ...planFor(sym, entry, sig) };
  { const cd = sig ? planCands(sig, {}).find(x => x.sym === sym && x.ok) : nu
  if (P.kind === 'manual' && !P.stop) {   // no tested plan today: suggest a
    const d = await loadDaily(sym);
    if (d && d.candles.length > 60) {
      const c = d.candles, i = c.length - 1, ind = L.indicators(c, null), a =
      P.stop = +Math.max(vol, trend || 0).toFixed(2); P.stopWhy = trend && tr
      if (P.stop > entry) P.stopWhy += '. It is above your entry, so it locks
      P.stopWhy += '. I will raise it if the stock rises, never lower it.';
    }
  }
  const st = num(/stop\s+(\d+(?:\.\d+)?)/i), tg = num(/target\s+(\d+(?:\.\d+)
  guideTargets(P); P.mktAtBuy = sig && sig.market ? (sig.market.above50 === f
  pos[sym] = P;
  if (shares > 0) { const pv = privRead(); pv.buys = (pv.buys || []).concat({
    const saved = privWrite(pv); out.push('Noted ' + shares + ' shares, about
  out.push('Tracking ' + posLine(sym, P) + (P.stopWhy ? '\n  Suggested stop '
} else if ((k = text.match(/^(?:i\s+)?sold\s+half\s+([a-z.\-]{1,6})(?:\s+(?:a
  const sym = k[1].toUpperCase(), P = pos[sym]; if (!P) { out.push('I was not
  let exit = k[2] ? +k[2] : null; if (!exit) { const j = await liveQ(sym); ex
  if (exit && P.entry) fs.appendFileSync(MY_TRADES, JSON.stringify({ sym, kin
  let news = ''; { const pv = privRead(); if (pv.shares && pv.shares[sym]) {
  P.trail = true; P.halfSince = ny(Date.now()).d; P.stop = Math.max(P.stop ||
  out.push('Half of ' + sym + ' sold' + (exit ? ' at ' + money(exit) + ' (' +
} else if ((k = text.match(/^(?:i\s+)?sold\s+([a-z.\-]{1,6})(?:\s+(?:at\s+)?(
  const sym = k[1].toUpperCase(), shTxt = (text.match(/(\d+)\s*shares?\b/i) |
  if (!pos[sym]) {   // already sold today (for example with the Sold button)
    const today = ny(Date.now()).d, rows = fs.existsSync(MY_TRADES) ? fs.read
    let ix = -1; for (let i = rows.length - 1; i >= 0; i--) { try { const r =
    if (ix < 0 || (!k[2] && !shSaid)) { out.push('I was not tracking ' + sym
    const r = JSON.parse(rows[ix]); if (k[2]) { r.exit = +k[2]; r.pct = +(r.e
    rows[ix] = JSON.stringify(r); fs.writeFileSync(MY_TRADES, rows.join('\n')
    let msg = sym + ': sale already recorded today. Updated with your price '
    const pv = privRead(), key = sym + '|' + today, was = (pv.soldUsd || {})[
    if (n && r.entry) { const usd = n * (r.exit - r.entry) - 2 * FEE() - n *
      addRealized(pv, sym, usd - prev, today, was ? 'correction' : undefined)
      msg += ' In money: about ' + (usd >= 0 ? '+' : '') + money(usd) + ' USD
    out.push(msg); continue; }
```

```
    let exit = k[2] ? +k[2] : null; if (!exit) { const j = await liveQ((sym));
    const P = pos[sym]; delete pos[sym];
    if (exit && P.entry) {     // your real trades are the evidence that matter
      const t = { sym, kind: P.kind || 'manual', entry: P.entry, exit, since: P
      // LOSS LEDGER: why each losing trade lost (gaps through the stop are the
      if (t.pct <= 0) t.lossType = P.stop && exit < P.stop * 0.995 ? 'gap throu
      fs.appendFileSync(MY_TRADES, JSON.stringify(t) + '\n');
      { const pv = privRead(), sh = (pv.shares && pv.shares[sym]) || shSaid; if
        else out.push('I do not know how many ' + sym + ' shares you had, so th
      out.push('Sold ' + sym + ' at ' + money(exit) + ' (' + (k[2] ? 'your pric
    } else out.push('Stopped tracking ' + sym + '. I could not get a price, so
  } else if ((k = text.match(/^quiet\s+(on|off)\s*$/i))) { const st = readJson(
  } else if (/^(themes?|sectors? flow|industr(y|ies)|where is the money)\s*\??\
  } else if (/^(skips?|my skips|skipped)\s*\??\s*$/i.test(text)) { out.push(awa
  } else if (/^(health|system health|status)\s*\??\s*$/i.test(text)) { const H
  } else if (/^(calendar|economic calendar|events this week|upcoming( events)?|
  } else if (/^(research|experiments?|registry|ideas tested)\s*\??\s*$/i.test(t
  } else if (/^(events?|news study|catalysts?|company news)\s*\??\s*$/i.test(te
  } else if (/^(day ?trades?|fast ?movers?|runners?|fast)\s*\??\s*$/i.test(text
  } else if (/^(news ?watch|my news|news alerts?|threats?)\s*\??\s*$/i.test(tex
  } else if (/^(paper|alpaca|paper account)\s*\??\s*$/i.test(text)) { out.push(
  } else if (/^(shadow|track ?record|forward)\s*\??\s*$/i.test(text)) { out.pus
  } else if (/^(days?|weekdays?|day of (the )?week|best day|monday)\s*\??\s*$/i
  } else if (/^(timing|when|when to (buy|sell|trade)|pre ?market|after ?hours|p
  } else if (/^(actions?|what to do|todo)\s*$/i.test(text)) { out.push(await ho
  } else if (/^(goals?|targets?|milestones?|my goals?|plan my goals|timeline)\s
  } else if ((k = text.match(/^(?:size|buy size|per buy)\s+\$?(\d+(?:\.\d+)?)\s
  } else if (/^budget\s+(open|off|none|no limit|unlimited)\s*$/i.test(text)) {
  } else if ((k = text.match(/^budget\s+\$?(\d+(?:\.\d+)?)\s*(?:usd|\$)?\s*$/i)
  } else if ((k = text.match(/^trades\s+(\d+)\s*(?:a month|per month|monthly)?\
  } else if ((k = text.match(/^pnl\s+([+-]?\d+(?:\.\d+)?)\s*([a-z.\-]{1,6})?\s*
    const pv = privRead(), usd = +k[1], sym = (k[2] || 'manual').toUpperCase();
    out.push(privWrite(pv) ? 'Recorded ' + (usd >= 0 ? '+' : '') + money(usd) +
  } else if ((k = text.match(/^(?:stops?\s+(?:placed|set|done|in)|placed\s+stop
    const list = k[1] ? [k[1].toUpperCase()] : Object.keys(pos); const done = [
    for (const sy of list) if (pos[sy]) { pos[sy].stopOk = { price: pos[sy].sto
    out.push(done.length ? 'Stop confirmed at your broker: ' + done.join(', ')
  } else if ((k = text.match(/^(stop|target)\s+([a-z.\-]{1,6})\s+(\d+(?:\.\d+)?
    const sym = k[2].toUpperCase(); if (!pos[sym]) { out.push('I am not trackin
    const mine = k[1].toLowerCase() === 'stop';
    if (mine) { pos[sym].stop = +k[3]; delete pos[sym].stopWhy; pos[sym].own =
    out.push('Updated ' + posLine(sym, pos[sym]) + (mine ? '\n  This is now you
  } else if (/^(positions?|status|list)\b/i.test(text)) {
    const syms = Object.keys(pos); out.push(syms.length ? 'Your tracked positio
  } else if (/^(help|commands?|\?)\s*$/i.test(text)) out.push(HELP);
  else out.push(await hotList(sig) + '\n\nYou do not need commands. Write anyth
}
```

```
  fs.writeFileSync('data/tg-state.json', JSON.stringify(state)); savePos(pos);
  if (out.length) await telegram(out.join('\n\n'));
  if (ups.length) console.log('Telegram: ' + ups.length + ' message(s) read');
}

// ---- Followed channels: public YouTube feeds of channels you choose. Every sto
// is logged under that channel's name, checked by the tested rules, and scored o
const YT = local(env.YT_BASE) || 'https://www.youtube.com', CH_FILE = 'data/chann
const readChannels = () => { const a = readJson(CH_FILE, []); return Array.isArra
const unxml = (t) => String(t || '').replace(/&amp;/g, '&').replace(/&lt;/g, '<')
const FUNDS = new Set(['SPY', 'QQQ', 'IWM', 'DIA', 'VOO', 'VTI', 'TQQQ', 'SQQQ',
function tickersIn(text, known) {
  const found = new Set(), add = (x) => { if (x && x.length > 1 && !NOT_TICKERS.h
  for (const m of text.matchAll(/\$([A-Z]{1,5})\b/g)) add(m[1]);
  for (const m of text.matchAll(/\b(?:NASDAQ|NYSE|AMEX)\s*:\s*([A-Z]{1,5})\b/g))
  for (const m of text.matchAll(/\(([A-Z]{2,5})\)/g)) add(m[1]);
  for (const m of text.matchAll(/\b[A-Z]{2,5}\b/g)) if (known.has(m[0])) add(m[0]
  return [...found].slice(0, 5);
}
// Reads a channel's newest videos. With a YT_KEY secret it uses YouTube's offici
// without one it falls back to the public feed, which YouTube often refuses to c
const YTAPI = local(env.YTAPI_BASE) || 'https://www.googleapis.com';
async function ytFeed(id) {
  if (env.YT_KEY) {
    try {
      const r = await fetch(YTAPI + '/youtube/v3/playlistItems?part=snippet&maxRe
      if (!r.ok) return { ok: false, status: r.status, via: 'YouTube Data API' };
      const items = (((await r.json()).items) || []).map(x => x.snippet || {}).ma
      return { ok: true, items, channel: (items[0] || {}).channel || '', via: 'Yo
    } catch (e) { return { ok: false, status: -1, via: 'YouTube Data API' }; }
  }
  const H = { 'User-Agent': 'Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (
  let status = 0;
  for (let t = 0; t < 2; t++) {
    try { const r = await fetch(YT + '/feeds/videos.xml?channel_id=' + id, { head
      if (r.ok) { const xml = await r.text(), items = xml.split('<entry>').slice(
        return { ok: true, items, channel: unxml((xml.match(/<author>\s*<name>([\
    await sleep(FAST ? 0 : 2500);
  }
  return { ok: false, status, via: 'public feed' };
}
async function channelMentions(known, now) {
  const out = [], chans = readChannels().slice(0, 12); let ok = 0;
  for (const ch of chans) {
    try {
      const f = await ytFeed(ch.id); if (!f.ok) { console.log('YouTube (' + f.via
      for (const e of f.items) {
```

```
        if (!(now - e.pub < 3 * 86400000)) continue;
        const title = e.title, desc = e.desc.slice(0, 1500);
        for (const sym of tickersIn(title + ' ' + desc, known)) if (!out.some(o =
      }
    } catch (e) {}
    await sleep(FAST ? 0 : 500);
  }
  return { mentions: out.slice(0, 12), channels: chans.length, ok };
}

// ---- Crowd radar: the stocks Reddit's trading forums mention most (ApeWisdom's
// each name is run through the same tested rules, only the ones with tested supp
const AW = local(env.AW_BASE) || 'https://apewisdom.io';
async function crowdList() {
  try {
    const r = await fetch(AW + '/api/v1.0/filter/all-stocks/page/1', { headers: {
    const skip = new Set(['SPY', 'QQQ', 'IWM', 'DIA', 'VOO', 'VTI', 'TQQQ', 'SQQQ
    const rows = (((await r.json()).results) || []).filter(x => /^[A-Z]{1,5}$/.te
    if (!rows.length) { console.log('crowd radar: the feed answered but no ticker
    const top = rows.slice(0, 5), surge = rows.filter(x => !top.includes(x) && x.
    return [...top, ...surge.map(x => ({ ...x, surge: true }))];
  } catch (e) { return null; }
}

// ---- LEADERS: ride the strongest stocks. The rule has no tuned settings: rank
// 50 and 200 day averages while the S&P 500 is above its own 200 day average, bu
// daily moves (ATR) under the close. It is tested on ALL stocks of the fixed lis
const LEAD_COST = 0.004;   // round trip: broker fee of 0.15% each way plus about
const LEAD_FILE = 'data/leaders.json', LEADCFG = { N: 5, LOOK: 126, ATR: 3 }, LEA
// SELF-TUNING. Nine versions of the one rule (strength measured over 3, 6 or 12
// re-tested every night. A version is usable only if it made money overall AND i
// when another is clearly better on the OLDER years, so the choice is never made
const LEAD_VARIANTS = [63, 126, 252].flatMap(LOOK => [2, 3, 4].map(ATR => ({ LOOK
const leadName = (v) => 'strength over ' + Math.round(v.LOOK / 21) + ' months, st
{ const k = (readJson0(LEAD_FILE) || {}).rule, v = LEAD_VARIANTS.find(x => leadKe
function leadersPick(loaded, spy, btSet, current) {
  const cut = spy[Math.max(0, spy.length - 505)].d, res = [];
  for (const v of LEAD_VARIANTS) {
    Object.assign(LEADCFG, v);
    const r = leadersCalc(loaded, spy, btSet), early = r.done.filter(x => x.d < c
    const mean = (a) => a.length ? a.reduce((x, y) => x + y, 0) / a.length : 0, m
    res.push({ key: leadKey(v), v, r, ts: early.length >= 20 && sd > 0 ? m / sd *
  }
  const ok = res.filter(x => x.ok).sort((a, b) => b.ts - a.ts), cur = res.find(x
  // The whole-market FAIR test (dead stocks included) outranks this nightly test
  // the past. While the pre-registered version (6 months, 3 daily moves) passes
  // not switch to a version the fair test has not passed.
```

```
  const FT = readJson0('data/leaders-study.json'), fr = FT && (FT.rows || []).fin
  const pinOk = Boolean(fb && fb.n >= 30 && fb.avg > 0 && (fb.pf || 0) > 1.2 && (
  const pick = pinned || (cur.ok && (!ok.length || ok[0].ts < cur.ts + 0.5) ? cur
  Object.assign(LEADCFG, pick.v);
  return { ...pick.r, rule: pick.key, was: cur.key, tried: res.length, passed: ok
}
function leadersCalc(loaded, spy, btSet) {
  const T = spy.length, tOf = new Map(spy.map((k, t) => [k.d, t])), spyE = L.indi
  for (const sym of Object.keys(loaded)) {
    if (sym === 'SPY' || FUNDS.has(sym)) continue;
    const c = loaded[sym].candles; if (c.length < 260) continue;
    const ind = L.indicators(c, null), at = new Int32Array(T).fill(-1);
    for (let i = 0; i < c.length; i++) { const t = tOf.get(c[i].d); if (t !== und
    S[sym] = { c, ind, at };
  }
  const elig = (sym, t) => {
    const x = S[sym], i = x.at[t]; if (i < 200) return null;
    const c = x.c, px = c[i].c, e50 = x.ind.e[50][i], e200 = x.ind.e[L.CFG.TREND]
    if (!(px >= 5) || !(a > 0) || !(px > e50 && e50 > e200)) return null;
    let dv = 0; for (let k = i - 19; k <= i; k++) dv += c[k].c * c[k].v; if (dv /
    if (i <= LEADCFG.LOOK) return null;
    const mom = px / c[i - LEADCFG.LOOK].c - 1; if (!(mom > 0)) return null;
    return { sym, i, mom, px, a, e50 };
  };
  const rank = (t, only) => Object.keys(S).filter(x => !only || only.has(x)).map(
  // the past test
  const open = [], done = [], t0 = Math.max(330, T - 2520);
  for (let t = t0; t < T - 1; t++) {
    for (let k = open.length - 1; k >= 0; k--) {
      const p = open[k], x = S[p.sym], i = x.at[t]; if (i < 0 || i < p.i0) contin
      const cl = x.c[i].c;
      if (cl < p.stop || cl < x.ind.e[50][i]) { if (i + 1 < x.c.length) { const e
      else p.stop = Math.max(p.stop, cl - LEADCFG.ATR * x.ind.atr[i]);
    }
    if (open.length < LEADCFG.N && spy[t].c > spyE[t]) for (const r of rank(t, bt
      if (open.length >= LEADCFG.N) break;
      if (open.some(p => p.sym === r.sym)) continue;
      const c = S[r.sym].c; if (r.i + 1 >= c.length) continue;
      const en = c[r.i + 1].o; if (en > r.px + 0.5 * r.a || en <= r.px - LEADCFG.
      const te = tOf.get(c[r.i + 1].d); if (te === undefined) continue;
      open.push({ sym: r.sym, entry: en, i0: r.i + 1, stop: r.px - LEADCFG.ATR *
    }
  }
  let bt = null;
  if (done.length >= 30) {
    done.sort((a, b) => a.d < b.d ? -1 : 1);
    const avg = (a) => a.length ? a.reduce((x, y) => x + y, 0) / a.length : 0, w
```

```
    const days = done.map(x => x.days).sort((a, b) => a - b), sp = done.filter(x
    const growth = done.reduce((g, x) => g * (1 + x.pct / LEADCFG.N), 1);
    bt = { n: done.length, from: spy[t0].d.slice(0, 4), win: w.length / done.leng
      spy: sp.length ? avg(sp.map(x => x.spy)) : null, first: avg(done.slice(0, h
      perYear: Math.pow(growth, 1 / years) - 1, spyYear: Math.pow(spy[T - 1].c /
  }
  return { S, bt, done, marketOk: spy[T - 1].c > spyE[T - 1], ranked: rank(T - 1,
}
const leadEvidence = (b) => !b ? 'Past test: too few trades to judge.' : 'Past te
  + (b.spy !== null ? ' (S&P 500 over the same days ' + (b.spy >= 0 ? '+' : '') +
  + '\nCaution: the test list is today's well-known companies, which flatters the
// The paper ledger's summary: what decides profit is the size of winners against
function leadLedger(dn) {
  const w = dn.filter(p => p.pct > 0), l = dn.filter(p => p.pct <= 0), sum = (a)
  let eq = 0, peak = 0, dd = 0; for (const p of dn) { eq += p.pct / LEADCFG.N; pe
  return ' Winners averaged ' + (w.length ? pc(sum(w) / w.length) : 'none yet') +
}
const leadLine = (r) => r.sym + ': BUY ' + money(r.buyLo) + ' to ' + money(r.buyH
const leadersText = () => { const s = readJson(LEAD_FILE, null); return s && s.te

async function daily() {
  const now = Date.now(), loaded = {}, errors = [];
  await tgCommands(readJson('data/signals.json', null));
  const myPos = loadPos(), myLines = [], posVerdict = {};
  for (const x of Object.keys(myPos)) if (!LIST.includes(x)) LIST.push(x);
  const crowd = env.NO_CROWD ? null : await crowdList();
  if (crowd) for (const x of crowd) if (!LIST.includes(x.sym)) LIST.push(x.sym);
  const chan = await channelMentions(new Set([...LIST, ...QEXTRA, ...DISC]), now)
  for (const x of chan.mentions) if (!LIST.includes(x.sym)) LIST.push(x.sym);
  for (const sym of LIST) {
    const j = await td('/time_series?symbol=' + encodeURIComponent(sym) + '&inter
    if (j.status === 'error' || !j.values) { errors.push(sym); continue; }
    const all = j.values.map(v => ({ d: v.datetime.slice(0, 10), o: +(+v.open).to
    loaded[sym] = { candles: all.filter(c => barClosed(c.d, now)), last: all[all.
  }
  const WIDE = new Set();
  { const t0 = Date.now(), ws = await wideUniverse(new Set(LIST));
    if (ws.length) { const n = await loadWide(ws, loaded, now); for (const x of w
  if (!loaded.SPY) { await telegram('Stock scanner: could not load market data to
  const spy = loaded.SPY.candles, session = spy[spy.length - 1].d, prev = readJso
  if (prev && prev.session === session && !env.FORCE) { console.log('No new sessi
  const mkt = L.marketMap(spy), mktUp = mkt[session].up, mom6 = spy[spy.length -
  const models = readJson('data/models.json', {});
  const journal = fs.existsSync('data/journal.jsonl') ? fs.readFileSync('data/jou
  const uni = [];
  const VR = L.volRegime ? L.volRegime(spy) : { known: false, mult: 1 }, VOLM = V
  const stocks = [], fresh = [], exits = [], openTrades = [], results = [], horiz
```

```
// LESSONS: every finished live paper trade is reviewed: how it ended, how far
// reached its first target. Grouped by the exact rule that produced it. A rule
// (10+ trades) is DEMOTED: its signals stay on paper and are no longer offered
const PREV_LESSONS = readJson('data/lessons.json', { rules: {} });
const weak = [];
const refLine = (sym, d) => {
  if (myPos[sym]) return;   // already held: it is tracked under YOUR POSITIONS
  const r = L.refPlan ? L.refPlan(d.candles, d.last) : null; if (!r) return;
  const o = L.refOdds ? L.refOdds(d.candles) : null;
  if (o && o.won < 0.25) { weak.push(sym + ' ' + Math.round(o.won * 100) + '%')
  picks.push({ tier: r.extended ? 3 : 2, text: sym + ' closed ' + money(d.last)
    + (o ? '\n  marker reached before the stop in ' + Math.round(o.won * 100) +
};

for (const sym of LIST) {
  const d = loaded[sym]; if (!d) continue;
  const only = QONLY.has(sym);
  if (d.candles.length < 600) { if (DISC.has(sym)) refLine(sym, d); continue; }
  const c = d.candles, i = c.length - 1, ind = L.indicators(c, mkt), T = L.tren
  if (!only && !WIDE.has(sym) && i > 140) { const e2 = ind.e[L.CFG.TREND], e5 =
  if (myPos[sym]) {   // your own position: end-of-day checks
    if (myPos[sym].stopWhy && c.some(k => k.d > myPos[sym].since)) { const a =
    if (myPos[sym].kind === 'leader' && myPos[sym].stop && c.some(k => k.d > my
    if (myPos[sym].trail && myPos[sym].stop) { const ns = +(c[i].c - 3 * ind.at
    const P = myPos[sym], held = c.filter(k => k.d > P.since).length, flags = [
    if (P.kind === 'leader' && held >= 1 && c[i].c < ind.e[50][i]) flags.push('
    if (P.kind === 'entry' && c[i].d >= P.since && !(P.stop && c[i].c <= P.stop
      const vv = !P.exitMA && P.key && L.VARIANTS ? L.VARIANTS.find(x => L.vkey
      if (lv) P.exitBelow = +lv.toFixed(2);
      if (vv && L.exitAt ? L.exitAt(ind, c, vv, i) : (lv && c[i].c < lv)) flags
    if (P.stop && c[i].c <= P.stop) flags.push('CLOSED AT OR BELOW YOUR STOP '
    if (held >= 1 && P.t1 && c[i].h >= P.t1) flags.push('Target 1 ' + money(P.t
    const gapT = P.q === 'gap', hit = gapT ? c[i].c < ind.sma5[i] : c[i].c > in
    if (P.trail) { if (c.filter(k => k.d > (P.halfSince || P.since)).length >=
    else if (P.kind === 'quick' && held >= 1 && hit) flags.push('QUICK TRADE EX
    else if ((P.kind === 'quick' || P.kind === 'radar' || (P.kind === 'entry' &
    { const xf = flags.find(f => /STOP|EXIT/.test(f)); if (xf) P.exitFlag = { d
    if (flags.some(f => /STOP|EXIT/.test(f))) posVerdict[sym] = flags.find(f =>
    { const bc = L.bearishCandle ? L.bearishCandle(c, i) : 0; if (bc) flags.pus
    if (P.raised === c[i].d) flags.push('Stop raised to ' + money(P.stop) + (P.
    myLines.push(posLine(sym, P, d.last) + (flags.length ? '\n  ' + flags.join(
  }
  const mine = journal.filter(x => x.sym === sym), resolved = mine.map(x => ({
  for (const y of resolved) {
    const isQ = y.x.kind === 'quick', si = c.findIndex(k => k.d === y.x.d);
    if (y.t.open) {
      openTrades.push({ sym, stop: +y.t.stop.toFixed(4), target: isFinite(y.t.t
```

```
    // Exit alert: the rule's exit fired at this close, so the paper trade is
    const timeUp = y.x.v.maxHold && i - si >= y.x.v.maxHold;
    if (i > si && (L.exitAt(ind, c, y.x.v, i) || timeUp)) exits.push(sym + (i
  } else {
    { const fk = isQ ? (y.x.v.q === 'gap' ? 'Quick trade: gap and hold' : y.x
    { const lr = live[y.x.key] || (live[y.x.key] = { name: (isQ ? 'Quick trad
      let mx = 0; for (let k = y.t.entryIdx; k <= y.t.exitIdx; k++) mx = Math
      { const hiB = y.x.ref + 0.5 * (y.x.dist / y.x.v.mult), above = y.t.entr
        if (above) { lr.fill.upN++; lr.fill.upSum += y.t.pct; } else { lr.fil
      lr.n++; lr.sum += y.t.pct; if (y.t.pct > 0) lr.wins++; lr.ends[y.t.reas
      if (y.x.t1) { lr.t1n++; if (mx >= y.x.t1 * (y.t.entry / y.x.ref)) lr.t1
      lr.last.push(sym + ' ' + (y.t.pct >= 0 ? '+' : '') + (y.t.pct * 100).to
    if (isQ) qFin.push({ d: c[y.t.exitIdx].d, pct: y.t.pct, spy: ind.bench &&
    if (y.t.exitIdx === i) exits.push(sym + (isQ ? ' (quick trade)' : '') + '
  }
  if (!isQ) { const o = L.barOutcomes(y.x, c, ind); for (const h in o) { cons
}
if (only) uni.push({ sym, last: d.last, uptrend: T.up, mom6: +T.mom6.toFixed(
if (!only) {
  const Lr = L.learn(c, ind, models[sym], sym === 'SPY'); models[sym] = Lr.ke
  if (Lr.proven || (Lr.test.n >= 15 && Lr.avgPct > 0)) addMday(Lr.mday);   //
  if (Lr.dow && (Lr.proven || (Lr.test.n >= 15 && Lr.avgPct > 0))) for (const
  // YOUR STOCKS: the sell target is re-tested every night on the same rule y
  if (myPos[sym] && !myPos[sym].ownT && myPos[sym].entry && (myPos[sym].kind
    let R = null; if (P.kind === 'entry' && (!P.key || P.key === Lr.key)) R =
    if (P.kind === 'quick' && L.quickStudy) { const qs = L.quickStudy(c, ind,
    if (R) { P.dist0 = P.dist0 || (P.stop && P.stop < P.entry ? +(P.entry - P
      const tp = R.v && R.v.tp, tg = R.tgt;
      if (tp && P.dist0) { P.t1 = +(P.entry + tp * P.dist0).toFixed(2); P.tmo
      else if (tg && tg.mode !== 'none' && tg.x) { P.t1 = +(P.entry * (1 + tg
      else { delete P.t1; delete P.t2; delete P.tmode; P.tnote = tg ? tg.verd
      if (P.orig && (P.t1 || null) !== (P.orig.t1 || null)) P.tnote = 'NEW TE
      P.tChecked = c[i].d; } }
  const paused = L.isPaused(resolved.filter(y => !y.t.open && y.x.kind !== 'q
  const v = L.applyGates(L.verdict(c, ind, Lr, null, d.last, paused), { dolla
  const row = { sym, last: d.last, state: v.word, why: v.why, halal, rule: L.
  if (v.act) Object.assign(row, { buyLo: d.last, buyHi: +(d.last + 0.5 * ind.
  stocks.push(row);
  if (BIOBLOCK.has(sym)) { if (v.act) { blocked.push(sym + ': a tested rule f
  else if (DISC.has(sym) && halal !== 'fail') {
    if (v.act) picks.push({ tier: 0, text: sym + ' closed ' + money(d.last) +
    else if (Lr.proven && trig.price && trig.price / d.last < 1.15) picks.pus
    else refLine(sym, d);
  }
  if (v.signal && !BIOBLOCK.has(sym) && !mine.some(x => x.d === c[i].d && x.k
}
// Quick trades: three kinds of dip, each shown only where it passed its own
```

```
  if (L.quickStudy && !myPos[sym] && !BIOBLOCK.has(sym) && d.last >= (WIDE.has(
    const QS = L.quickStudy(c, ind, sym === 'SPY'); for (const q of QS || []) a
    if (Q) {
      const qd = Q.v.mult * ind.atr[i], per10k = Math.floor(VOLM * Math.min(100
      const t1 = P.t1 ? +P.t1.toFixed(4) : null, t2 = null, qhalf = !!P.half, t
      quick.push({ t: Q.test.t, row: { sym, last: d.last, buyLo: d.last, buyHi:
        text: sym + ' closed ' + money(d.last) + '\n  why: ' + (Q.v.q === 'gap'
      quick[quick.length - 1].entry = mine.some(x => x.d === c[i].d && x.kind =
    }
  }
}
quick.sort((a, b) => b.t - a.t);
{ const days = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday'], avgOf
  const allO = avgOf(DOWS.older), allN = avgOf(DOWS.newer), rows = days.map((d,
    return { day: d, older: { n: o[0], avg: ao }, newer: { n: w[0], avg: an },
  fs.writeFileSync('data/dow-study.json', JSON.stringify({ session, all: { olde
  { const tags = ['bigDown', 'down', 'flat', 'up', 'bigUp', 'gapDown', 'gapFlat
    const rows = tags.map(k => { const o = MDAY.older[k] || [0, 0], w = MDAY.ne
      return { k, older: { n: o[0], avg: ao }, newer: { n: w[0], avg: an }, ver
    fs.writeFileSync('data/mday-study.json', JSON.stringify({ session, all, n:
  const av = (x) => x[0] ? x[1] / x[0] : null, T2 = {}; for (const h of ['older
  const better = (a, b) => ['older', 'newer'].every(h => T2[h][a].n >= 150 && T
  T2.buy = better('closeIn', 'openIn') ? 'CLOSE' : better('openIn', 'closeIn')
  fs.writeFileSync('data/timing-study.json', JSON.stringify({ session, ...T2 },
// EARNINGS DRIFT: a stock that beat its EPS estimate by 10%+ and whose reactio
// Offered only while the Sunday fair test (earnings-study.mjs) says PROMISING;
const pead = []; { const ES = readJson0('data/earnings-study.json'), ER = readJ
  const nextDay = (d) => { let ms = Date.parse(d + 'T12:00:00Z'); do { ms += 86
  for (const r of (ER && ER.reports) || []) { const d = loaded[r.sym]; if (!d |
    const c = d.candles, i = c.length - 1, b = c[i], p = c[i - 1]; if (b.d !==
    const react = r.t === 'bmo' ? r.d : r.t === 'amc' ? nextDay(r.d) : null; if
    if (!(Math.abs(r.e) >= 0.05)) continue; const sur = (r.a - r.e) / Math.abs(
    if (sur < R.minSurprise || up < R.minUp || loc < R.minLoc || !(b.c >= 5) ||
    let s20 = 0; for (let k = i - 19; k <= i; k++) s20 += c[k].c;
    pead.push({ sym: r.sym, last: b.c, buyLo: +b.c.toFixed(2), buyHi: +(b.c * (
// EVENT DRIFT: company news (an SEC filing) that the Sunday study (events-stud
// closed up strongly on it today. Only the stocks that jumped today are checke
// offered while no news kind has passed. Earnings days are left to the earning
const evt = []; { const ES = readJson0('data/events-study.json'), CK = readJson
  const ok = ES && ES.fam ? Object.keys(ES.fam).filter(k => ES.fam[k].verdict =
  if (ok.length) { const jumped = [];
    for (const [sym, d] of Object.entries(loaded)) { if (!d || d.candles.length
      const c = d.candles, i = c.length - 1, b = c[i], p = c[i - 1]; if (b.d !=
      const up = b.c / p.c - 1, loc = b.h > b.l ? (b.c - b.l) / (b.h - b.l) : 0
      if (up < (R.minUp || 0.03) || loc < (R.minLoc || 0.5) || !(b.c >= (R.minP
      let s20 = 0; for (let k = i - 19; k <= i; k++) s20 += c[k].c; jumped.push
    jumped.sort((x, y) => y.up - x.up); let asked = 0;
```

```
    const add = (x, k, p) => { const F = ES.fam[k], t = F.results && F.results.
      evt.push({ sym: x.sym, fam: k, name: F.name.replace(/ \(.*\)$/, ''), p, l
    for (const x of jumped) { if (!fam8.length || asked >= 120 || !CK[x.sym]) c
      const j = await secJson(SEC_DATA + '/submissions/CIK' + String(CK[x.sym][
      let earn = false; const hits = [];
      for (let k = 0; k < Math.min(r.form.length, 40); k++) { const pt = String
        const f = evFams(r.form[k], (r.items || [])[k], j.sic, ES); if (f.inclu
      if (!earn && hits.length) add(x, hits[0][0], hits[0][1]); }
    // earnings growth speeding up: tonight's earnings reactions, checked again
    if (ok.includes('EPS_ACCEL')) { let EC = {}; try { EC = JSON.parse(zlib.gun
      const ER = readJson0('data/earnings-recent.json');
      for (const r of (ER && ER.reports) || []) { const x = jumped.find(y => y.
        const H = (EC[r.sym] || []).filter(q => isFinite(q.a) && q.d <= r.d), n
        if (n < 6 || H[n - 1].d !== r.d) continue; const g1 = g(n - 1), g0 = g(
    if (evt.length) console.log('event drift tonight: ' + evt.map(e => e.sym +

// Earnings check (Finnhub). Results within 7 days block a quick trade or a con
const earnings = async (sym) => {
  if (!env.FINNHUB_KEY) return { known: false };
  try {
    const today = new Date(now).toISOString().slice(0, 10), r = await fetch(FH
    if (!r.ok) return { known: false };
    const cal = ((await r.json()).earningsCalendar) || [], ds = cal.map(e => e.
    return { known: true, date: ds[0] || null, days: ds[0] ? Math.round((Date.p
  } catch (e) { return { known: false }; } finally { await sleep(FAST ? 0 : 110
};
const earnText = (e) => !e.known ? 'earnings date NOT CHECKED, verify before ac
const keep = [];
for (const q of quick.slice(0, 10)) {
  const e = await earnings(q.row.sym);
  if (e.known && e.date && e.days <= 7) { blocked.push(q.row.sym + ' (quick tra
  const nr = await newsRisk(q.row.sym);
  if (nr.hard) { blocked.push(q.row.sym + ' (quick trade): ' + nr.hard.tag + '
  q.text += '\n  ' + earnText(e) + (nr.soft ? '\n  WARNING, ' + nr.soft.tag.toL
  if (q.entry) { fresh.push(q.entry); fs.appendFileSync('data/journal.jsonl', J
  keep.push(q);
}
quick.length = 0; quick.push(...keep);
// QUICK-TRADE GATE (stronger quick trades). The dip family loses on the whole
// 1) If Sunday's lab found a condition under which dips work on the whole mark
// 2) If no condition passed and the persistence test says a stock's own record
// 3) Otherwise (or before the lab has run): a strict gate, 25+ unseen trades a
// Every dip is still logged on paper, so the evidence keeps growing.
const quickHeld = []; {
  const HL = readJson0('data/hunting-lab.json'), best = HL && HL.quickFilters ?
  const moms = stocks.map(x => x.mom6).filter(x => typeof x === 'number').sort(
  const s50 = spy.slice(-50).reduce((a, x) => a + x.c, 0) / Math.min(50, spy.le
```

```
  const meets = (r) => !best || best.split('+').every(k => k === 'up' ? up50 :
  const strictOk = (r) => { const d = distOf(r.pcts, r.last); return (r.n || 0)
  const pass = [];
  for (const q of quick) { const r = q.row;
    const why = best ? (meets(r) ? null : 'does not meet the condition that wor
    if (why) quickHeld.push({ sym: r.sym, why }); else pass.push(q); }
  quick.length = 0; quick.push(...pass); }
for (const st of stocks.filter(x => x.state === 'Buy watch')) {
  const e = await earnings(st.sym);
  if (e.known && e.date && e.days <= 7) { st.state = 'Blocked'; blocked.push(st
  const nr = await newsRisk(st.sym);
  if (nr.hard) { st.state = 'Blocked'; blocked.push(st.sym + ': a tested rule f
  else { st.earn = earnText(e) + (nr.soft ? '\n  WARNING, ' + nr.soft.tag.toLow
}
// ---- LEADERS: rank tonight, keep a paper portfolio of five going forward, an
// memory: the whole-market stocks have been tested already; keep only their la
// re-test (which runs nine versions over every loaded stock and would not fit
for (const x of WIDE) if (loaded[x]) loaded[x] = { candles: loaded[x].candles.s
const coreLoaded = Object.fromEntries(Object.entries(loaded).filter(([k]) => !W
const LS0 = readJson(LEAD_FILE, { paper: [], done: [] }), LS = LS0, LC = leader
LS.paper = (LS.paper || []).filter(p => !(p.picked === session && !p.entry)); L
for (const p of LS.paper) {
  const x = LC.S[p.sym]; if (!x) continue; const c = x.c, i = c.length - 1;
  if (!p.entry) { const k = c.findIndex(z => z.d > p.picked); if (k >= 0) { if
  if (p.exitOn && !p.exit) { const k = c.findIndex(z => z.d > p.exitOn); if (k
  else if (p.entry && !p.exitOn) { if (c[i].c < p.stop || c[i].c < x.ind.e[50][
}
LS.done.push(...LS.paper.filter(p => p.exit)); LS.done = LS.done.slice(-300);
LS.paper = LS.paper.filter(p => !p.exit && !p.skip);
// The weekly fair test (whole market, dead stocks included, judged only on dat
const FAIR = readJson('data/leaders-study.json', null), fairRow = FAIR && (FAIR
const rowRejected = Boolean(fairRow && fairRow.test && fairRow.test.base && fai
const fairBad = Boolean(FAIR && ((FAIR.liveVerdict === 'REJECTED' && FAIR.live
const fairState = !FAIR ? 'NOT RUN' : fairBad ? 'REJECTED' : FAIR.live === LC.r
const leadOk = LC.bt && LC.bt.avg > 0 && LC.bt.second > 0 && !fairBad;
if (LC.marketOk && (leadOk || fairBad)) {
  let checked = 0;
  for (const r of LC.ranked) {
    if (LS.paper.length >= LEADCFG.N || checked >= 12) break;
    if (LS.paper.some(p => p.sym === r.sym)) continue;
    checked++;
    if (BIOBLOCK.has(r.sym)) { leadSkip.push(r.sym + (BIOBLOCK.get(r.sym).start
    if (myHalal(r.sym) === 'fail') { leadSkip.push(r.sym + ' (you marked it NOT
    const e = await earnings(r.sym); if (e.known && e.date && e.days <= 7) { le
    const nr = await newsRisk(r.sym); if (nr.hard) { leadSkip.push(r.sym + ' ('
    const row = { sym: r.sym, last: r.px, buyLo: +r.px.toFixed(2), buyHi: +(r.p
    if (leadOk) leadRows.push(row); LS.paper.push({ sym: r.sym, picked: session
```

```
  }
}
{
  const pc = (x) => (x >= 0 ? '+' : '') + (x * 100).toFixed(1) + '%', top = LC.
  leadOut.push('LEADERS, ' + session + ' close', 'The strongest stocks I follow
  if (fairBad) leadOut.push('NO PRICES: the weekly fair test on the whole marke
  else if (!leadOk) leadOut.push('NO PRICES: on the past test this rule did not
  else if (!LC.marketOk) leadOut.push('NO NEW BUYS: the S&P 500 is under its 20
  else if (leadRows.length) { leadOut.push('NEW TO BUY (' + leadRows.length + '
  else leadOut.push('NEW TO BUY: none tonight. All five places are taken by the
  const lastOf = (p) => loaded[p.sym] ? ', closed ' + money(loaded[p.sym].last)
  if (sells.length) leadOut.push('', 'SELL AT THE NEXT OPEN (' + sells.length +
  if (held.length) leadOut.push('', 'STILL RIDING (' + held.length + ')', ...he
  if (leadSkip.length) leadOut.push('', 'Strong but left out: ' + leadSkip.join
  leadOut.push('', 'Rule in use: ' + leadName(LEADCFG) + '.' + (LS0.rule && LC.
    FAIR && FAIR.verdict && FAIR.verdict !== 'NOT RUN' ? 'FAIR TEST (whole mark
  const dn = LS.done.filter(p => typeof p.pct === 'number');
  leadOut.push('Paper record since this list started: ' + (dn.length ? dn.lengt
  LS.session = session; LS.text = leadOut.join('\n'); LS.bt = LC.bt; LS.rule =
  fs.writeFileSync(LEAD_FILE, JSON.stringify(LS, null, 1));
}
// Circuit breaker: if the last five finished quick paper trades lost money ove
// Setups are still journaled as paper trades, so the record keeps running and
qFin.sort((a, b) => a.d < b.d ? -1 : 1);
const lastFive = qFin.slice(-5), quickPaused = lastFive.length >= 5 && lastFive
const tipRows = readTips().map(t => { const dd = loaded[t.sym], k = dd ? dd.can
  return k < 0 || ks < 0 ? { sym: t.sym, src: t.src, d: t.d, ret: null, spy: nu
const bySrc = {}; for (const t of tipRows) if (t.ret !== null) { const g = bySr
// Self-pruning: once a source has a real record (10+ tips, a month old) and it
// Its mentions are still logged and scored, so it can earn its way back, but t
const srcStats = Object.values(bySrc).map(g => ({ ...g, avg: g.avg / g.n, spy:
const MUTED = new Set(srcStats.filter(g => g.muted).map(g => g.src)), TRUSTED =
const tipBoardData = { sources: srcStats, tips: tipRows };
savePos(myPos);
const LESSONS = { updated: new Date(now).toISOString(), session, rules: {} };
for (const [k, r] of Object.entries(live)) {
  const med = [...r.runUp].sort((a, b) => a - b)[Math.floor(r.runUp.length / 2)
  LESSONS.rules[k] = { name: r.name, n: r.n, win: r.wins / r.n, avg, ends: r.en
    wasDemoted: Boolean(PREV_LESSONS.rules && PREV_LESSONS.rules[k] && PREV_LES
}
fs.writeFileSync('data/lessons.json', JSON.stringify(LESSONS, null, 1));
for (const r of [...quick.map(x => x.row), ...stocks.filter(s => s.state === 'B
{ const SECF = 'data/sectors.json', SC = readJson(SECF, {}); let asked = 0;
  for (const r of [...quick.slice(0, 10).map(x => x.row), ...stocks.filter(s =>
    if (SC[r.sym] === undefined && env.FINNHUB_KEY && asked < 15) { asked++; tr
    r.sector = SC[r.sym] || null;
  }
```

```
  fs.writeFileSync(SECF, JSON.stringify(SC)); }
const qn = qFin.length, qs = qFin.filter(x => x.spy !== null);
const quickRecord = qn ? { n: qn, winRate: qFin.filter(x => x.pct > 0).length /

const fin = results.length, wins = results.filter(r => r > 0).length;
const outcomes = { updated: new Date(now).toISOString(), finished: fin, winRate
  horizons: Object.fromEntries(Object.entries(horizons).map(([h, g]) => [h, { n
fs.writeFileSync('data/outcomes.json', JSON.stringify(outcomes, null, 1));
fs.writeFileSync('data/models.json', JSON.stringify(models, null, 1));
{ const n = await zoyaCheck([...quick.slice(0, 30).map(x => x.row.sym), ...stoc
{ const n = await halalExtCheck([...quick.slice(0, 30).map(x => x.row.sym), ...
fs.writeFileSync('data/signals.json', JSON.stringify({ generatedAt: new Date(no
fs.writeFileSync('data/alert-state.json', JSON.stringify({ day: '', sent: {} })
try { await alpacaPaper(readJson('data/signals.json', null), loaded, session);

const buys = stocks.filter(s => s.state === 'Buy watch'), near = stocks.filter(
const msg = ['Daily scan, ' + session + ' close', 'All prices below are from th
for (const s of buys) msg.push(s.sym + (DISC.has(s.sym) ? ' (discovery candidat
if (!buys.length) msg.push('None today.');
else msg.push('Manual review only. Check earnings dates and halal status before
if (quickPaused) msg.push('', 'QUICK TRADES PAUSED BY THE CIRCUIT BREAKER', 'Th
else if (quick.length) msg.push('', 'QUICK TRADES (' + Math.min(quick.length, 3
else msg.push('', 'QUICK TRADES', 'No setup at this close.');
for (const x of Object.keys(myPos)) if (!myLines.some(l => l.startsWith(x + ':'
if (myLines.length) msg.splice(3, 0, '', 'YOUR POSITIONS (' + myLines.length +
if (crowd) {     // only names with tested support are shown; the rest are coun
  const good = [], dropped = [], old = readTips(), day = session;
  for (const x of crowd) {
    const st = stocks.find(k => k.sym === x.sym), q = quick.find(k => k.row.sym
    if (loaded[x.sym] && !old.some(o => o.sym === x.sym && o.src === 'Reddit cr
    if (BIOBLOCK.has(x.sym) || blocked.some(b => b.startsWith(x.sym + ':') || b
    else if (st && st.state === 'Buy watch') good.push(tag + ': ENTRY CONFIRMED
    else if (q) good.push(tag + ': a tested quick trade setup fired. See QUICK
    else if (st && st.proven && st.trigger && st.trigger / st.last < 1.15) good
    else dropped.push(x.sym);
  }
  if (MUTED.has('Reddit crowd')) msg.push('', 'CROWD RADAR', 'Muted: over its r
  else msg.push('', 'CROWD RADAR (Reddit\u2019s most-mentioned stocks, filtered
}
if (chan.mentions.length) {
  const good = [], dropped = [], old = readTips();
  for (const x of chan.mentions) {
    const st = stocks.find(k => k.sym === x.sym), q = quick.find(k => k.row.sym
    if (!loaded[x.sym]) continue;                                    // not a re
    if (!old.some(o => o.sym === x.sym && o.src === src && Date.parse(session)
    if (MUTED.has(src)) { if (!dropped.includes('(' + x.name + ' is muted: its
    else if (BIOBLOCK.has(x.sym)) dropped.push(x.sym + ' (blocked)');
```

```
    else if (st && st.state === 'Buy watch') good.push(tag + ': ENTRY CONFIRMED
    else if (q) good.push(tag + ': a tested quick trade setup fired. See QUICK
    else if (st && st.proven && st.trigger && st.trigger / st.last < 1.15) good
    else dropped.push(x.sym + ' (' + x.name + ')');
  }
  msg.push('', 'FOLLOWED CHANNELS (stocks named in the last 3 days, filtered by
}
if (blocked.length) msg.push('', 'BLOCKED, EVENT OR NEWS RISK (' + blocked.leng
if (exits.length) msg.push('', 'SELL / REASSESS WATCH (paper trades)', ...exits
if (near.length) { msg.push('', 'BUY WATCH, WAITING FOR CONFIRMATION'); near.fo
if (picks.length) { picks.sort((a, b) => a.tier - b.tier); msg.push('', 'DISCOV
if (weak.length) msg.push('', 'NO PLAN, POOR ODDS (' + weak.length + ')', 'The
if (quickRecord) msg.push('', 'Quick trade scoreboard: ' + quickRecord.n + ' fi
if (fin) msg.push('', 'Paper record: ' + fin + ' finished, ' + Math.round(wins
if (FMOVERS.length) msg.push('', 'TODAY\'S FAST MOVERS, VERDICT', ...FMOVERS.ma
msg.splice(1, 0, ...conclusion(readJson('data/signals.json', null), myPos), '',
// Scan health: what was asked for, what came back, and which feeds were off. I
const fm = readJson('data/fm-state.json', {}), health = ['SCAN HEALTH', 'Prices
  'Earnings check (Finnhub): ' + (env.FINNHUB_KEY ? 'on' : 'OFF, no key') + '.
  'Crowd radar (ApeWisdom): ' + (env.NO_CROWD ? 'off' : crowd ? 'on, ' + crowd.
  'Followed channels: ' + (chan.channels ? chan.ok + ' of ' + chan.channels + '
  'Discovery list: ' + (DISCMETA && DISCMETA.session ? 'from the ' + DISCMETA.s
// OPTIONS DATA PROBE (read-only, one request): does the Massive key include op
if (env.MASSIVE_KEY) { let st = 'no answer', fields = []; try { const r = await
    if (r.ok) { const j = await r.json(), o = j && j.results && j.results[0]; i
  fs.writeFileSync('data/options-probe.json', JSON.stringify({ at: new Date().t
  health.push('Options data (Massive): ' + (fields.length ? 'AVAILABLE with ' +
if (errors.length > LIST.length * 0.2) health.push('WARNING: more than a fifth
msg.push('', ...health);
fs.writeFileSync('data/scan-report.txt', msg.join('\n') + '\n\n' + leadOut.join
const healthBad = errors.length > LIST.length * 0.2;
// ---- Weekly review: sent with the scan of each Friday close (or when WEEKLY
if (!QUIET() && (new Date(session + 'T12:00:00Z').getUTCDay() === 5 || env.WEEK
  const pc = (x) => (x >= 0 ? '+' : '') + (x * 100).toFixed(1) + '%', si = spy.
  const w = ['Weekly review, week ending ' + session, 'S&P 500 fund: ' + pc(spy
  const bySec = {}; for (const x of wk) { const k = SEC[x.sym]; if (!k || k ===
  const secs = Object.entries(bySec).sort((a, b) => b[1].chg / b[1].n - a[1].ch
  if (secs.length) w.push('', 'SECTORS, strongest first', ...secs.map(([k, g])
  const sorted = [...wk].sort((a, b) => b.chg - a.chg);
  if (sorted.length >= 6) w.push('', 'BIGGEST MOVES ON YOUR LIST', 'Up: ' + sor
  const better = wk.filter(x => x.up && !x.wasUp).map(x => x.sym), worse = wk.f
  w.push('', 'TREND CHANGES', 'Moved into an uptrend: ' + (better.join(', ') ||
  const weekStart = new Date(Date.parse(session) - 6 * 86400000).toISOString().
  w.push('', 'SIGNALS THIS WEEK', newSig.length ? newSig.length + ' logged. Qui
  const fams = Object.entries(famRes);
  if (fams.length) w.push('', 'WHAT IS WORKING, all finished paper trades', ...
  else w.push('', 'WHAT IS WORKING', 'No paper trade has finished yet.');
```

```
    if (DISCMETA && DISCMETA.candidates) { const bl = DISCMETA.candidates.filter(
    if (tipBoardData.sources.length) w.push('', 'TIP SOURCES', ...tipBoardData.so
    w.push('', 'FOR NEXT WEEK', 'Check earnings dates and halal status on the pag
    await telegram(w.filter(x => x !== '').join('\n').replace(/\n(SECTORS|BIGGEST
  }
  // ---- The last message of the night. Quiet mode (the default): ONE action mes
  if (QUIET()) { const sg = readJson('data/signals.json', null);
    await telegram(await actionsText(sg, loadPos(), null, 'ACTIONS FOR THE NEXT S
  else {
  // ---- The last message of the night: one short plan. Everything above it is b
  { const sg = readJson('data/signals.json', null);
    await telegram(planText(sg, loadPos(), null, 'PLAN FOR THE NEXT SESSION, from
      + (healthBad ? '\n\nWARNING: over a fifth of the prices failed to load toni
      + '\n\n' + myRecord() + '\nNightly report: send  report   Your record: send
    // every night: one clear action for each stock you own (sell all, sell part,
    if (Object.keys(loadPos()).length) { try { await telegram(await holdingsText(
    // every Friday close: the full HOLD OR SELL study of every stock you own, se
    const mine = Object.keys(loadPos());
    if (mine.length && (new Date(session + 'T12:00:00Z').getUTCDay() === 5 || env
      const parts = []; for (const x of mine.slice(0, 10)) { try { parts.push(awa
      if (parts.length) await telegram('WEEKLY REVIEW OF YOUR STOCKS (automatic,
  }
  console.log('daily scan done:', stocks.length, 'stocks,', buys.length, 'buy wat
}

async function intraday(onlyMine) {
  const now = Date.now();
  const sig = readJson('data/signals.json', null);
  await tgCommands(sig);
  if (env.BOT_ONLY) return;   // fast reply loop: answer messages only; the full
  await preMarket(sig);
  if (onlyMine) { await fastMovers(); await watchdog(sig); }
  if (onlyMine && (!Object.keys(loadPos()).length || new Date(now).getUTCMinutes(
  if (!marketOpen(now) && !env.FORCE) { console.log('Market closed. Nothing to do
  if (!sig) { console.log('No daily scan yet.'); return; }
  const myPos = loadPos();
  const st = readJson('data/alert-state.json', { day: '', sent: {} }), day = ny(n
  const OKI = new Set(planCands(sig, loadPos()).filter(x => x.ok).map(x => x.sym)
  const qk = sig.quick || [], planBuys = planCands(sig, myPos).filter(x => x.ok);
  const buyNowCards = [];
  const watch = onlyMine ? Object.keys(myPos).slice(0, 12) : [...new Set([...Obje
  const seen = {};
  for (const sym of watch) {
    const j = await liveQ((sym)), p = +j.price; if (!(p > 0)) continue; seen[sym]
    const say = (k, text) => { if (!st.sent[sym + k]) { st.sent[sym + k] = 1; ale
    const b = buys.find(s => s.sym === sym), t = trig.find(s => s.sym === sym), q
    if (mp) {   // you told the bot you bought this one
```

```
    const chg = ' You bought at ' + money(mp.entry) + ' (' + (p >= mp.entry ? '
    if (mp.stop && p <= mp.stop) say('myst', 'YOUR POSITION: STOP LOSS REACHED
    else if (mp.t2 && p >= mp.t2) say('myt2', 'YOUR POSITION: TARGET 2 REACHED
    else if (mp.t1 && p >= mp.t1 && !mp.trail) say('myt1', 'YOUR POSITION: ' +
  }
  const pb = mp ? null : planBuys.find(x => x.sym === sym);
  if (pb && fresh && p >= pb.lo && p <= pb.hi && !st.sent[sym + 'buynow']) { st
  const qs = mp || (pb && st.sent[sym + 'buynow']) ? null : qs0;
  if (qs) {   // a quick trade setup from last night's close
    if (p > qs.buyHi) say('qhi', 'QUICK TRADE: above its entry range (' + money
    else if (p <= qs.stop) say('qlo', 'QUICK TRADE: already at its invalidation
    else if (p < qs.buyLo) say('qlow', 'QUICK TRADE: below its planned entry ra
    else say('qin', 'QUICK TRADE: trading inside its entry range (' + money(qs.
  }
  const ld = mp ? null : (sig.leaders || []).find(s => s.sym === sym);
  if (ld) {   // a leader picked at last night's close
    if (p <= ld.stop) say('lst', 'LEADER: already at its stop ' + money(ld.stop
    else if (p > ld.buyHi) say('lhi', 'LEADER: above its buy range (' + money(l
    else say('lin', 'LEADER: in its buy range now (' + money(ld.buyLo) + ' to '
  }
  if (b && p > b.buyHi) say('hi', 'above its buy range (' + money(b.buyLo) + '
  if (b && p <= b.stop) say('lo', 'already down to its stop level ' + money(b.s
  if (t && p >= t.trigger) say('tr', 'BUY WATCH forming. It is above its trigge
  for (const o of (sig.openTrades || []).filter(x => x.sym === sym)) {
    if (p <= o.stop) say('st' + o.since, 'SELL / REASSESS WATCH. Paper trade fr
    if (o.t1 && p >= o.t1) say('t1' + o.since, 'QUICK TRADE: paper trade from '
    if (o.target && p >= o.target) say('tg' + o.since, 'paper trade from ' + o.
  }
}
// ONE MARKET-OPEN CHECK a day, so you always know the system is running and wh
let openMsg = null;
if (!onlyMine && !st.sent.__open && Object.keys(seen).length) {
  st.sent.__open = 1;
  const kt = new Date().toLocaleTimeString('en-GB', { timeZone: 'Asia/Kuwait',
  const posL = Object.entries(myPos).filter(([k]) => seen[k]).map(([k, P]) => {
    return k + ' ' + money(p) + ' (' + (g >= 0 ? '+' : '') + (g * 100).toFixed(
  const buyL = planBuys.filter(x => seen[x.sym] && !myPos[x.sym]).map(x => { co
  openMsg = ['MARKET OPEN CHECK (' + kt + ' Kuwait). The system is running.', '
}
fs.writeFileSync('data/alert-state.json', JSON.stringify(st));
if (openMsg) await telegram(openMsg + '\n' + FOOT);
alerts.sort((a, b) => (b.includes('YOUR POSITION') ? 1 : 0) - (a.includes('YOUR
for (const bc of buyNowCards) { const cd = planCands(sig, myPos).find(x => x.sy
  await telegram('BUY NOW: a proven setup is inside its buy range (' + new Date
    [[{ text: 'Bought ' + bc.sym + (sh ? ' (' + sh + ' sh at ~' + money(bc.p) +
if (alerts.length) await telegram('Market hours check\n\n' + alerts.join('\n\n'
console.log('intraday check done:', watch.length, 'watched,', alerts.length, 'n
```

```
}

await (MODE === 'chat' ? intraday(true) : MODE === 'intraday' ? intraday() : dail
```